

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
МЕЖИНСТИТУТСКАЯ БАЗОВАЯ КАФЕДРА

КУРСОВОЙ ПРОЕКТ

по дисциплине

«Программная инженерия»

на тему:

**«Разработка кроссплатформенного клиентского приложения для сети
Matrix»**

Выполнил: Иванов Дмитрий Романович
студент 3 курса
группы ПИЖ-б-о-23-2(1)
направления подготовки 09.03.04 «Программная инженерия»
направленность (профиль)
«Разработка и сопровождение программного обеспечения»
очной формы обучения

(подпись)

Руководитель проекта: Самойлов Ф.В., доцент межинститутской базовой
кафедры

Работа допущена к защите

(подпись

(дата)

руководителя)

Работа выполнена и защищена
с оценкой

Дата защиты

(подпись)

(дата)

Члены комиссии:

доцент межинститутской базовой
кафедры

Самойлов Ф.В.

(подпись)

доцент межинститутской базовой
кафедры

Альбекова З.М.

(подпись)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

УТВЕРЖДАЮ

Зав. межинститутской базовой
кафедрой, канд. физ.-мат. наук

Е.Н. Новикова
«____» _____ 2026 г.

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Кафедра межинститутская базовая
Направление подготовки (специальность) 09.03.04 «Программная инженерия»
Направленность (профиль)/ специализация Разработка и сопровождение
программного обеспечения

ЗАДАНИЕ

на курсовой проект

студента

Иванова Дмитрия Романовича

по дисциплине «Программная инженерия»

1. **Тема проекта:** Разработка кроссплатформенного клиентского приложения для сети Matrix

2. **Целью выполнения курсового проекта является:**

– проектирование, программная реализация, тестирование и развертывание кроссплатформенной клиент-серверной системы управления профилем пользователя и групповыми чатами для мессенджера на базе открытого протокола Matrix с применением современных архитектурных шаблонов (PCMEF, Ports & Adapters) и технологического стека (Spring Boot, React, PostgreSQL).

3. **Задачи:**

– Выполнить детальный анализ предметной области, формализовать описание бизнес-процессов (в нотации IDEF0) и провести сравнительный анализ существующих аналогов для выявления конкурентных преимуществ разрабатываемой системы.

– Разработать формальную модель функциональных требований к проектируемому программному обеспечению на основе методологии RUP (Rational Unified Process), включая построение диаграммы прецедентов (Use Case), спецификацию ключевых сценариев использования и составление расширенного глоссария предметной области.

– Спроектировать доменную модель (Domain Model) системы и формализовать бизнес-правила, регулирующие процессы управления профилями, публикациями и групповым взаимодействием.

– Выполнить высокоуровневое архитектурное проектирование на основе пятислойного паттерна PCMEF (Presentation-Control-Mediator-Entity-Foundation)

– Спроектировать реляционную базу данных для хранения учетных записей, постов, комментариев, связей сообществ и пользовательских сессий, осуществить ее нормализацию до третьей нормальной формы (3НФ) и подготовить DDL-скрипты.

– Провести детальное проектирование динамического поведения системы с помощью UML-диаграмм последовательности (Sequence Diagrams), спроектировать структуру классов и обосновать применение паттернов проектирования GoF (Gang of Four).

– Реализовать серверную часть программной системы на языке Java с использованием платформы Spring Boot, обеспечив надежную обработку бизнес-логики, транзакционность и кэширование данных.

– Разработать и провести модульное (unit), интеграционное и ручное системное тестирование системы, измерить покрытие кода тестами и оценить производительность под нагрузкой.

– Произвести оценку трудоемкости разработки по модели COCOMO и сформировать инструменты управления проектом (WBS, диаграмма Ганта, реестр рисков).

4. Перечень подлежащих разработке вопросов:

а) по теоретической части

1. Провести анализ литературных источников по теме курсового проекта, включая периодику и интернет-ресурсы;

2. Сформулировать цели и задачи разработки;

3. Постановка условия задачи, выбор и описание методов и библиотек для ее решения и технологий, среды программирования.

б) по проектной части

1. Составить алгоритмы для решения поставленных задач;

2. Провести проектирование и моделирование процессов, интерфейса, используя методологии IDEF и UML;

в) по практической части

1. Провести разработку backend web-приложения с использованием фреймворка Spring Boot;

2. Провести разработку frontend web-приложения с использованием React и интеграцией с Tauri;

3. Провести отладку и тестирование разработанного программного продукта;

4. Привести описание файловой и функциональной структур web-приложения разработки;

5. Провести описание настройки рабочего окружения разработки и описание файлов проекта.

5. Исходные данные:

по вариантам, разработанным преподавателем

6. Список рекомендуемой литературы

1. Gamma, Erich. Design Patterns: Elements of Reusable Object-Oriented Software / Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. — Addison-Wesley Professional, 1995. — 395 p. — ISBN 0-201-63361-2.

2. Fowler, Martin. Patterns of Enterprise Application Architecture / Martin Fowler. — Addison-Wesley Professional, 2002. — 576 p. — ISBN 0-321-12742-0.

3. Evans, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software / Eric Evans. — Addison-Wesley Professional, 2004. — 560 p. — ISBN 0-321-12521-5.

4. Boehm, Barry W. Software Engineering Economics / Barry W. Boehm. — Prentice-Hall, 1981. — 768 p. — ISBN 0-13-822122-7.

7. Контрольные сроки представления отдельных разделов курсового проекта:

— 25 % - подготовка теоретической части курсового проекта «06» марта 2026 г.

— 50 % - подготовка проектной части курсового проекта «03» апреля 2026 г.

— 75 % - подготовка практической части курсового проекта «01» мая 2026 г.

– 100 % - оформление и подготовка пояснительной записки курсового проекта «22» мая 2026 г.

8. Срок защиты студентом курсового проекта

«30» мая 2026 г.

Дата выдачи задания «09» февраля 2026 г.

Руководитель курсового
проекта
доцент межинститутской базовой
кафедры

Задание принял к исполнению
студент очной формы обучения
3 курса группы ПИЖ-б-о-23-2(1)

Самойлов Ф.В.

Иванов Д.Р.

РЕФЕРАТ

Отчёт 77 с., 24 рис., 12 табл., 18 ист., 3 прил.

СОЦИАЛЬНАЯ СЕТЬ, МЕССЕНДЖЕР, ПРОТОКОЛ MATRIX, JAVA, SPRING BOOT, REACT, REST API, PCMEF

В пояснительной записке описана разработка системы управления профилем пользователя и групповыми чатами для мессенджера на базе протокола Matrix. В аналитической части проведён анализ предметной области, бизнес-процессов, конкурентов и обоснована необходимость разработки. В проектной части представлены модель требований, модель предметной области, архитектурное проектирование на основе паттерна PCMEF с адаптацией под гексагональные принципы, проектирование базы данных и детальное проектирование с применением паттернов GoF. В реализационной части описана разработка бизнес-логики, пользовательского интерфейса на React, REST API с применением OpenAPI, системы безопасности и транзакций. В разделе тестирования приведены результаты модульного, интеграционного и системного тестирования. В разделе развёртывания описана контейнеризация с использованием Docker и развёртывание в Kubernetes через Helm. В разделе управления проектом представлены WBS, диаграмма Ганта, оценка трудозатрат по модели COCOMO и управление рисками.

СОДЕРЖАНИЕ

Перечень сокращений и условных обозначений	10
Введение	12
1 Аналитическая часть	16
1.1 Описание предметной области	16
1.2 Анализ бизнес-процессов (IDEF0)	16
1.3 SWOT-анализ текущего состояния	18
1.4 SWOT-анализ стратегической рамки	19
1.5 Анализ аналогов и конкурентов	20
1.6 Обоснование необходимости разработки	21
1.7 Экономическое обоснование (ROI)	21
2 Проектная часть	23
2.1 Модель требований к ПО	23
2.1.1 Диаграмма прецедентов (Use Case)	23
2.1.2 Спецификация прецедентов	24
2.1.3 Глоссарий терминов	27
2.2 Модель предметной области	28
2.2.1 Архитектура слоев приложения и модель классов проектирования	28
2.2.2 Описание сущностей и атрибутов	29
2.2.3 Бизнес-правила	30
2.3 Архитектурное проектирование	31
2.3.1 Выбор архитектурного стиля	31
2.3.2 Диаграмма пакетов (PCMEF)	32
2.3.3 Описание слоёв и их ответственности	33
2.4 Проектирование базы данных	36
2.4.1 ER-диаграмма	36

2.4.2	Физическая модель данных и нормализация (1НФ, 2НФ, 3НФ)	36
2.4.3	DDL-скрипты	37
2.5	Детальное проектирование	37
2.5.1	Диаграммы последовательности	37
2.5.2	Диаграммы классов проектирования	40
2.5.3	Применение паттернов GoF	45
3	Реализационная часть	47
3.1	Реализация бизнес-логики	47
3.1.1	Структура проекта	47
3.1.2	Классы-сущности (Entity Layer)	47
3.1.3	Слой доступа к данным (Foundation Layer)	49
3.1.4	Слой бизнес-логики (Mediator Layer)	50
3.2	Рефакторинг и оптимизация	51
3.2.1	Статический анализ кода	51
3.2.2	Применение паттернов (Data Mapper, Identity Map)	51
3.2.3	Оптимизация SQL-запросов	51
3.3	Пользовательский интерфейс	52
3.3.1	Настольное (десктопное) приложение	52
3.3.2	Веб-приложение	52
3.3.3	Сравнение реализаций	52
3.4	Безопасность и транзакции	53
3.4.1	Аутентификация и авторизация	53
3.4.2	Управление транзакциями	54
3.4.3	Защита от атак и Rate Limiting	54
3.5	REST API	55
3.5.1	Спецификация OpenAPI	55

4	Тестирование и обеспечение качества	56
4.1	Модульное тестирование	56
4.2	Интеграционное тестирование	56
4.3	Системное тестирование	57
4.4	Нагрузочное тестирование	58
4.5	Результаты тестирования	59
5	Развёртывание и эксплуатация	60
5.1	Контейнеризация (Docker)	60
5.2	Локальный запуск (Docker Compose)	60
5.3	Оркестрация (Kubernetes)	61
5.4	Инструкция по установке и запуску	62
5.5	Инструкция по настройке параметров	63
5.6	Требования к окружению	63
6	Управление проектом	64
6.1	WBS (иерархическая структура работ)	64
6.2	Диаграмма Ганта	64
6.3	Оценка трудозатрат по модели COCOMO	65
6.4	Управление рисками	66
	Заключение	68
	Список использованных источников	70
	Приложение А Листинги кода	72
	Приложение Б DDL-скрипты базы данных	73
	Приложение В Скриншоты пользовательского интерфейса	75

Перечень сокращений и условных обозначений

ACID	Atomicity, Consistency, Isolation, Durability — атомарность, согласованность, изолированность, долговечность (требования к транзакционной системе)
API	Application Programming Interface — интерфейс программирования приложений
ADR	Architecture Decision Record — запись об архитектурном решении
BUC	Business Use Case — бизнес-прецедент
CRUD	Create, Read, Update, Delete — создание, чтение, обновление, удаление
CSS	Cascading Style Sheets — каскадные таблицы стилей
CSRF	Cross-Site Request Forgery — межсайтовая подделка запроса
CORS	Cross-Origin Resource Sharing — совместное использование ресурсов между разными источниками
DDD	Domain-Driven Design — предметно-ориентированное проектирование
DDL	Data Definition Language — язык описания данных
DOM	Document Object Model — объектная модель документа
DTO	Data Transfer Object — объект передачи данных
ER	Entity-Relationship — сущность-связь
GoF	Gang of Four — «банда четырёх», авторы классических паттернов проектирования
HTML	HyperText Markup Language — язык разметки гипертекста
HTTP	HyperText Transfer Protocol — протокол передачи гипертекста
IDEF0	Integration Definition for Function Modeling — методология функционального моделирования
JPA	Java Persistence API — интерфейс сохранения объектов Java
JRE	Java Runtime Environment — среда выполнения Java
JSON	JavaScript Object Notation — текстовый формат обмена данными
JWT	JSON Web Token — токен доступа в формате JSON
KLOC	Kilo Lines of Code — тысяча строк исходного кода
LTS	Long Term Support — долгосрочная поддержка программного обеспечения
MIME	Multipurpose Internet Mail Extensions — многоцелевые расширения интернет-почты (стандарт описания типов файлов)
OAuth	Open Authorization — открытый протокол авторизации
OIDC	OpenID Connect — протокол идентификации поверх OAuth 2.0
ORM	Object-Relational Mapping — объектно-реляционное отображение
OWASP	Open Worldwide Application Security Project — открытый проект обеспечения безопасности приложений
PCMEF	Presentation-Control-Mediator-Entity-Foundation — паттерн многослойной архитектуры
REST	Representational State Transfer — архитектурный стиль взаимодействия компонентов распределённого приложения
RPS	Requests Per Second — запросов в секунду
SDK	Software Development Kit — комплект средств разработки
SLOC	Source Lines of Code — строки исходного кода
SPA	Single Page Application — одностраничное веб-приложение
SPOF	Single Point of Failure — единая точка отказа

SQL	Structured Query Language — язык структурированных запросов
SSE	Server-Sent Events — события, отправляемые сервером
SSO	Single Sign-On — единая точка входа
SWOT	Strengths, Weaknesses, Opportunities, Threats — анализ сильных и слабых сторон, возможностей и угроз
TTL	Time To Live — время жизни записи в кэше
UC	Use Case — прецедент использования
UI	User Interface — пользовательский интерфейс
URL	Uniform Resource Locator — единый указатель ресурса
WBS	Work Breakdown Structure — иерархическая структура работ
XSS	Cross-Site Scripting — межсайтовый скриптинг
XML	eXtensible Markup Language — расширяемый язык разметки
ЗНФ	Третья нормальная форма — уровень нормализации реляционной базы данных
ОЗУ	Оперативное запоминающее устройство
ROI	Return on Investment — возврат инвестиций

ВВЕДЕНИЕ

В современном информационном обществе средства мгновенного обмена сообщениями (мессенджеры) превратились из вспомогательного инструмента коммуникации в ключевой элемент повседневной и деловой активности. Согласно статистическим исследованиям аналитического агентства Statista [1], в 2025 году глобальная аудитория пользователей мобильных мессенджеров перешагнула отметку в 3,5 миллиарда человек, а к 2029 году прогнозируется устойчивый рост до 4,5 миллиарда. В условиях столь агрессивной конкуренции на рынке программного обеспечения разработчики вынуждены непрерывно расширять функциональные возможности своих продуктов, сближая концепции классического мессенджера и социальной сети. Это приводит к интеграции таких элементов социального взаимодействия, как публичные профили, персональные ленты публикаций (посты), системы древовидных комментариев и тематические сообщества.

В то же время, доминирующие на рынке коммерческие платформы (такие как Telegram, VK, Discord, WhatsApp) строятся на основе централизованной архитектуры. Подобный подход влечет за собой ряд системных недостатков и рисков для конечных пользователей и корпоративного сектора:

- **Монополизация и уязвимость данных:** все персональные сведения, переписка и медиафайлы хранятся на серверах единого провайдера, что повышает риски утечки конфиденциальной информации в результате целевых кибератак или несанкционированного доступа.

- **Цензурирование контента:** владельцы централизованных платформ имеют возможность в одностороннем порядке модерировать, скрывать или безвозвратно удалять информацию, блокировать учетные записи без объяснения причин.

- **Отсутствие алгоритмической прозрачности:** алгоритмы формирования лент новостей и рекомендаций скрыты от общественности, что позволяет манипулировать вниманием пользователей.

- **Единая точка отказа (SPOF):** технический сбой в инфраструктуре центрального провайдера мгновенно парализует работу миллионов пользователей по всему миру.

В качестве альтернативы централизованным решениям активно развиваются децентрализованные федеративные протоколы. Одним из наиболее технологически зрелых стандартов является протокол Matrix [2]. Это открытый стандарт для защищенной децентрализованной коммуникации в реальном времени. В основе Matrix лежит федеративная модель: пользователи

регистрируются на различных независимых серверах (домашних серверах — homeservers), которые синхронизируют историю комнат между собой посредством криптографически защищенного протокола федерации.

Однако стандартные спецификации Matrix и их базовые серверные реализации (например, Synapse, Dendrite, Conduit) ориентированы преимущественно на обеспечение мгновенного обмена сообщениями в чатах и не предоставляют инструментов для построения социальных сетей. Существующие децентрализованные социальные сети общего назначения (например, Mastodon или Lemmy на базе протокола ActivityPub) функционируют как обособленные платформы, имеют слабую интеграцию с мессенджерами и сложны для развертывания и администрирования внутри корпоративного контура.

В связи с этим возникает явная научно-практическая потребность в разработке специализированной программной системы управления профилями пользователей и групповыми чатами, глубоко интегрированной с инфраструктурой Matrix. Это позволит:

- расширить базовые возможности Matrix-коммуникации без ущерба для принципов децентрализации и безопасности;
- предоставить пользователям и корпорациям полный контроль над их персональными данными в рамках федеративной сети;
- создать единую экосистему, объединяющую функции оперативного общения и социального обмена информацией.

Цель курсового проекта — проектирование, программная реализация, тестирование и развертывание кроссплатформенной клиент-серверной системы управления профилем пользователя и групповыми чатами для мессенджера на базе открытого протокола Matrix с применением современных архитектурных шаблонов (PCMEF, Ports & Adapters) и технологического стека (Spring Boot, React, PostgreSQL).

Для достижения поставленной цели необходимо решить комплекс следующих научно-практических задач:

1. Выполнить детальный анализ предметной области, формализовать описание бизнес-процессов (в нотации IDEF0) и провести сравнительный анализ существующих аналогов для выявления конкурентных преимуществ разрабатываемой системы.
2. Разработать формальную модель функциональных требований к проектируемому программному обеспечению на основе методологии RUP (Rational Unified Process), включая построение диаграммы прецедентов (Use

Case), спецификацию ключевых сценариев использования и составление расширенного глоссария предметной области.

3. Спроектировать доменную модель (Domain Model) системы и формализовать бизнес-правила, регулирующие процессы управления профилями, публикациями и групповым взаимодействием.

4. Выполнить высокоуровневое архитектурное проектирование на основе пятислойного паттерна PCMEF (Presentation-Control-Mediator-Entity-Foundation) с адаптацией под функционально-ориентированную (feature-first) структуру пакетов и принципы гексагональной архитектуры (Ports & Adapters).

5. Спроектировать реляционную базу данных для хранения учетных записей, постов, комментариев, связей сообществ и пользовательских сессий, осуществить ее нормализацию до третьей нормальной формы (3НФ) и подготовить DDL-скрипты.

6. Провести детальное проектирование динамического поведения системы с помощью UML-диаграмм последовательности (Sequence Diagrams), спроектировать структуру классов и обосновать применение паттернов проектирования GoF (Gang of Four).

7. Реализовать серверную часть программной системы на языке Java с использованием платформы Spring Boot, обеспечив надежную обработку бизнес-логики, транзакционность и кэширование данных.

8. Разработать клиентское веб-приложение на базе библиотеки React с использованием типизации TypeScript, встроив его в кроссплатформенную десктопную оболочку и реализовав интеграцию с сервером через REST API.

9. Реализовать надежную систему безопасности, основанную на федеративной аутентификации Matrix OpenID, авторизации по протоколу JWT с механизмом ротации токенов, а также защиту от распространенных веб-атак.

10. Документировать программный интерфейс REST API в соответствии со спецификацией OpenAPI 3.1.

11. Разработать и провести модульное (unit), интеграционное и ручное системное тестирование системы, измерить покрытие кода тестами и оценить производительность под нагрузкой.

12. Подготовить конфигурационные файлы контейнеризации (Docker), оркестрации (Kubernetes) и шаблонизации развертывания (Helm) для обеспечения жизненного цикла эксплуатации ПО.

13. Произвести оценку трудоемкости разработки по модели COCOMO и сформировать инструменты управления проектом (WBS, диаграмма Ганта, реестр рисков).

Объект исследования — процессы и технологии проектирования и разработки децентрализованных систем социального взаимодействия в информационно-коммуникационных сетях.

Предмет исследования — методы, архитектурные шаблоны и программные средства реализации компонентов управления профилями пользователей, публикациями и групповыми чатами в интеграции с федеративным протоколом Matrix с использованием Java/Spring Boot и TypeScript/React.

1 Аналитическая часть

1.1 Описание предметной области

Предметная область разрабатываемой системы охватывает функциональное пространство социальных медиаплатформ и систем мгновенного обмена сообщениями, интегрированных на стыке децентрализованного протокола Matrix [2]. В рамках данной системы выделяются следующие ключевые сущности и понятия:

- **Профиль пользователя** — логическая область, содержащая набор верифицированных и редактируемых персональных данных (отображаемое имя, уникальный Matrix ID, история аватаров, текстовый статус, дата рождения, адрес проживания, а также ссылка на медиаресурс с любимым аудио-треком), предназначенная для позиционирования пользователя в сети.

- **Публикация (пост)** — информационный блок, размещаемый в ленте профиля пользователя, который может содержать текстовый контент, ссылки и прикрепленные медиафайлы различных типов (изображения, аудиоматериалы, видеофайлы).

- **Комментарий** — текстовое сообщение, привязанное к конкретной публикации и выражающее мнение пользователя. Комментарии поддерживают иерархическую (древовидную) структуру посредством ссылки на родительский комментарий (`parent_id`).

- **Групповой чат** — виртуальное коммуникационное пространство в рамках протокола Matrix, объединяющее множество участников для обмена сообщениями в реальном времени.

- **Алиас (роль)** — локальное отображаемое имя (псевдоним) или функциональная роль пользователя, действующая исключительно в границах конкретного группового чата.

- **Сообщество** — организационное объединение пользователей и групповых чатов на основе общих интересов или организационной структуры предприятия, идентифицируемое уникальным тегом.

1.2 Анализ бизнес-процессов (IDEF0)

Для формализации функциональной структуры системы и определения информационных потоков было выполнено моделирование бизнес-процессов по методологии IDEF0.

На контекстной диаграмме верхнего уровня (A-0) основным процессом является «Функционирование системы управления профилем и групповыми чатами». На вход процесса поступают:

- запросы пользователей на чтение и запись (создание постов, редактирование профилей, написание комментариев);

- данные аутентификации (внешний Matrix OpenID токен);

- файлы медиаконтента (изображения, аудиофайлы, видео).

Управление процессом осуществляется на основе:

- правил и ограничений спецификации протокола Matrix;

- внутренних политик безопасности приложения и бизнес-правил

валидации контента.

Механизмами выполнения процесса выступают:

- программный сервер на базе Spring Boot;

- реляционная база данных PostgreSQL;

- клиентское веб-приложение на базе React.

Выходом процесса являются:

- актуализированные данные профилей и публикации;

- push-уведомления и рассылки для подписчиков в реальном времени;

- статистика активности пользователей.

При декомпозиции контекстного процесса (диаграмма A0) выделяются четыре ключевых функциональных блока:

1. **Блок A1: Аутентификация и управление сессиями** — отвечает за проверку подлинности пользователя через федеративное API Matrix и выпуск внутренней пары JWT-токенов.

2. **Блок A2: Ведение профиля пользователя** — обеспечивает CRUD-операции над персональными данными, хранение истории аватаров и управление приватностью.

3. **Блок A3: Управление социальным контентом** — реализует публикацию постов в ленту, загрузку медиафайлов и построение иерархических веток комментариев.

4. **Блок A4: Администрирование групп и сообществ** — обеспечивает управление членством, назначение ролей (алиасов) и фильтрацию контента.

Business Use Case — msocial (Matrix Social Network)

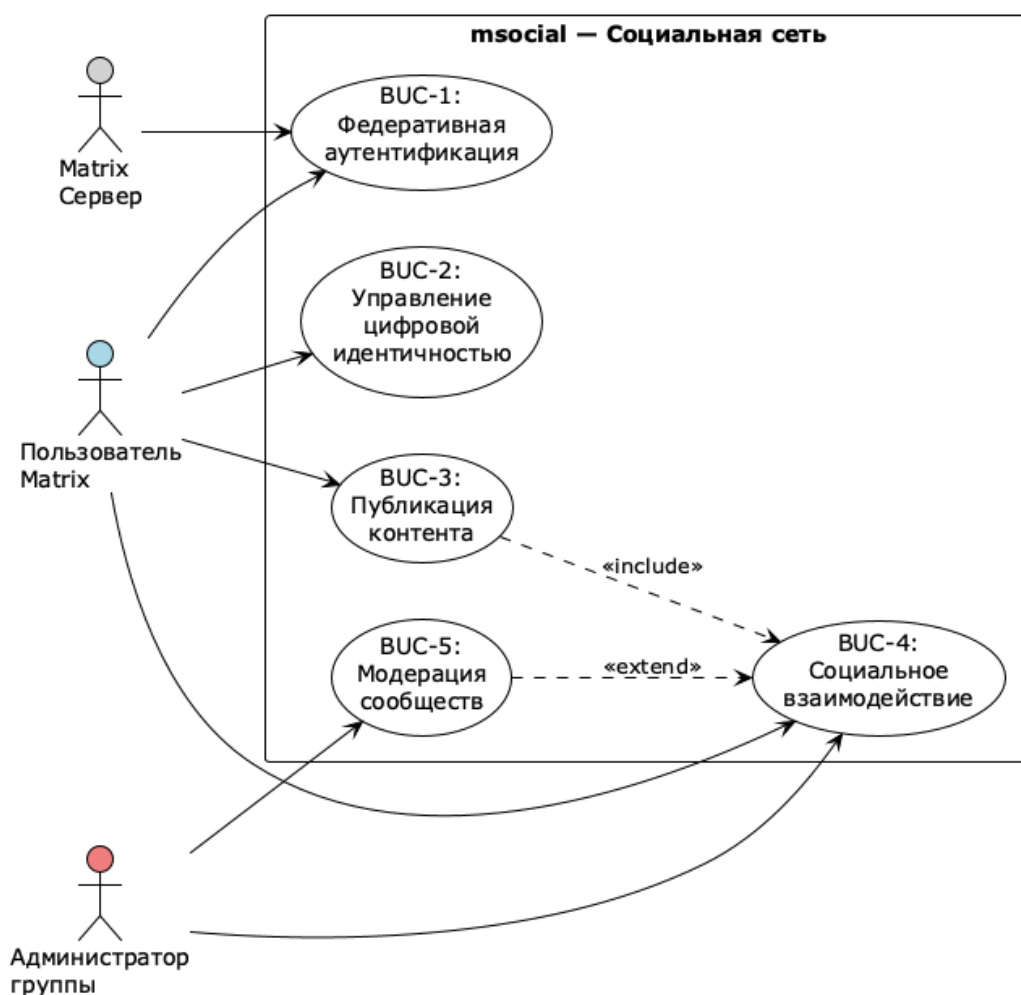


Рисунок 1 — Диаграмма бизнес-прецедентов (BUC)

1.3 SWOT-анализ текущего состояния

Для оценки стратегических перспектив внедрения разрабатываемой системы в экосистему Matrix был проведен SWOT-анализ, результаты которого представлены в таблице ниже.

Таблица 1 — SWOT-анализ системы на базе протокола Matrix

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
– Полная интеграция с открытым стандартом Matrix;n-Децентрализованная (федеративная) топология;n-Высокий уровень контроля пользователя над персональными	– Высокий порог входа для рядовых пользователей;n-Недостаточная развитость клиентского ПО по сравнению с коммерческими монополиями;n-Зависимость от производительности домашних серверов (homeservers).

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
данными;п- Возможность сквозного шифрования коммуникаций.	

На основе матрицы SWOT-анализа сформированы следующие стратегические направления развития:

- **S-O стратегия (использование сил для реализации возможностей):** активное продвижение системы в корпоративном секторе как безопасной альтернативы Slack и MS Teams, не зависящей от зарубежных облачных провайдеров.
- **W-O стратегия (минимизация слабостей за счет возможностей):** создание интуитивно понятного веб-интерфейса и автоматизированных скриптов развертывания для упрощения администрирования.
- **S-T стратегия (использование сил для защиты от угроз):** применение федеративной природы Matrix для обеспечения высокой отказоустойчивости системы в условиях блокировок централизованных сервисов.
- **W-T стратегия (сокращение слабостей и защита от угроз):** оптимизация кэширования и запросов к базе данных для минимизации требований к аппаратному обеспечению серверов.

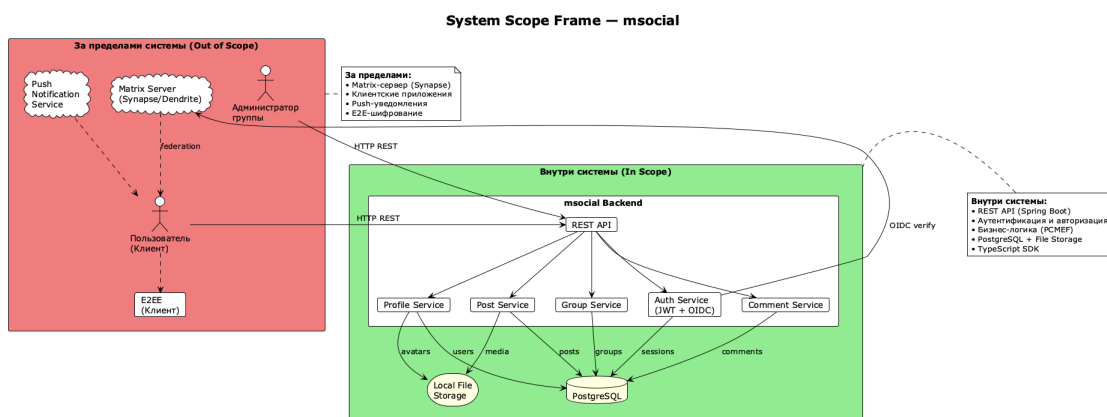


Рисунок 2 — Стратегическая рамка системы

1.4 SWOT-анализ стратегической рамки

Анализ стратегической рамки показывает, что разрабатываемая система фокусируется на создании высокой ценности в области приватности, безопасности и гибкости настройки под нужды конкретной организации. В отличие от крупных централизованных платформ, система не пытается максимизировать виральный охват за счет рекламных алгоритмов, а

концентрируется на предоставлении чистого социального пространства, контролируемого самими пользователями.

1.5 Анализ аналогов и конкурентов

Для определения рыночного позиционирования системы проведено сравнительное исследование существующих решений по ключевым критериям.

Таблица 2 — Сравнительный анализ аналогов и конкурентов

Продукт	Архитектура	Социальные функции	Открытость
Telegram	Централизованная	Каналы, публичные комментарии, боты, истории	Частично (только клиентская часть)
Discord	Централизованная	Тематические серверы, каналы, роли, интеграции	Нет (закрытый код)
Mastodon	Федеративная (ActivityPub)	Микроблоги, подписки, новостные ленты	Да (AGPL)
Element (Matrix)	Федеративная (Matrix)	Групповые чаты, комнаты обсуждений	Да (Apache 2.0)
Разрабатываемая система	Федеративная (Matrix)	Профили, посты, комментарии, алиасы, сообщества	Да (Открытый исходный код)

Детальный анализ конкурентов выявляет следующие закономерности:

1. **Telegram** и **Discord** предлагают развитый социальный функционал, но вынуждают пользователей доверять свои данные коммерческим компаниям без возможности развернуть приватный узел сети.

2. **Mastodon** решает проблему децентрализации, однако протокол ActivityPub не предназначен для мгновенного обмена сообщениями в чатах, что затрудняет построение единого коммуникационного пространства.

3. **Element (Matrix)** является мощным мессенджером, но в нем полностью отсутствуют привычные социальные функции, такие как персональные блоги пользователей или ленты новостей сообществ.

Разрабатываемая система ликвидирует этот разрыв, предлагая полноценные социальные возможности непосредственно внутри Matrix-сети, сохраняя совместимость с существующей инфраструктурой серверов.

1.6 Обоснование необходимости разработки

Создание проектируемой системы обусловлено следующими факторами:

1. **Дефицит децентрализованных социальных платформ:** существующие решения либо централизованы, либо сложны для интеграции с повседневными инструментами общения.

2. **Рост требований к корпоративной безопасности:** организациям необходим полный контроль над информационными потоками. Хранение переписки и профилей сотрудников на серверах третьих лиц недопустимо.

3. **Синергетический эффект Matrix:** использование готового транспорта сообщений и механизма комнат Matrix позволяет избежать написания сложного кода синхронизации данных с нуля.

4. **Масштабируемость архитектуры:** применение паттерна PCMEF позволяет системе развиваться без необходимости тотального рефакторинга.

1.7 Экономическое обоснование (ROI)

Оценка экономической эффективности внедрения системы произведена на примере гипотетической организации численностью 100 сотрудников. Сравнивается использование платного тарифа Slack Enterprise и развертывание собственного решения на базе разрабатываемого ПО.

Показатель возврата инвестиций (ROI) рассчитывается по классической формуле:

$$ROI = \left(\frac{\text{Экономия} - \text{Затраты}}{\text{Затраты}} \right) \times 100\% \quad (1)$$

Срок окупаемости (Payback Period, PP) определяется как:

$$PP = \frac{\text{Затраты}}{\frac{\text{Экономия}}{12}} \quad (2)$$

Таблица 3 — Экономическое обоснование разработки системы

Показатель	Значение	Примечание
Стоимость лицензий Slack Enterprise (100 пользователей/год)	\$15 000	Внешние годовые затраты
Стоимость инфраструктуры и поддержки системы (год)	\$8 000	Внутренние затраты (серверы, администрирование)

Показатель	Значение	Примечание
Экономия в первый год эксплуатации	\$7 000	Разница в стоимости владения
ROI (первый год)	87,5%	Расчет по формуле выше
Срок окупаемости	13,7 месяцев	Период до выхода в плюс

Расчеты демонстрируют высокий экономический потенциал внедрения разрабатываемой системы. Начиная со второго года эксплуатации, экономический эффект становится еще более выраженным, так как первоначальные затраты на интеграцию и настройку программного обеспечения нивелируются.

2 Проектная часть

2.1 Модель требований к ПО

2.1.1 Диаграмма прецедентов (Use Case)

Диаграмма вариантов использования (прецедентов) определяет функциональные границы разрабатываемой системы и взаимодействия между ее пользователями и внутренними компонентами. В системе выделяются три роли действующих лиц (акторов):

- **Пользователь (User)** — авторизованный участник системы, который взаимодействует с социальными функциями (публикация постов, комментирование, управление своим профилем) и создает личные группы.
- **Администратор группы (Group Administrator)** — пользователь, наделенный расширенными полномочиями в рамках конкретного группового чата Matrix. Он осуществляет управление участниками, переопределяет их отображаемые имена и назначает локальные алиасы (роли).
- **Система (System)** — автоматизированный бэкенд-компонент, функционирующий в фоновом режиме. Система осуществляет валидацию данных, автоматическое сопоставление тегов сообществ, ведение истории изменений и фоновую синхронизацию с серверами Matrix.

В рамках системы определены десять основных прецедентов использования (вариантов использования):

- **Управление профилем пользователя:** изменение глобального имени и статуса.
- **Управление публикациями:** создание, редактирование и удаление постов.
- **Комментирование:** написание отзывов к публикациям.
- **Управление персональной информацией:** редактирование даты рождения, адреса, любимого трека.
- **История аватаров:** загрузка новых аватаров и просмотр истории предыдущих.
- **Управление группами:** создание и настройка групповых пространств.
- **Назначение алиасов:** присвоение ролей участникам групп администраторами.
- **Изменение имени участника:** переопределение имени пользователя в рамках чата.
- **Тег сообщества:** автоматическое форматирование имени при публикациях.

– **Список сообществ:** просмотр объединений, в которых состоит пользователь.

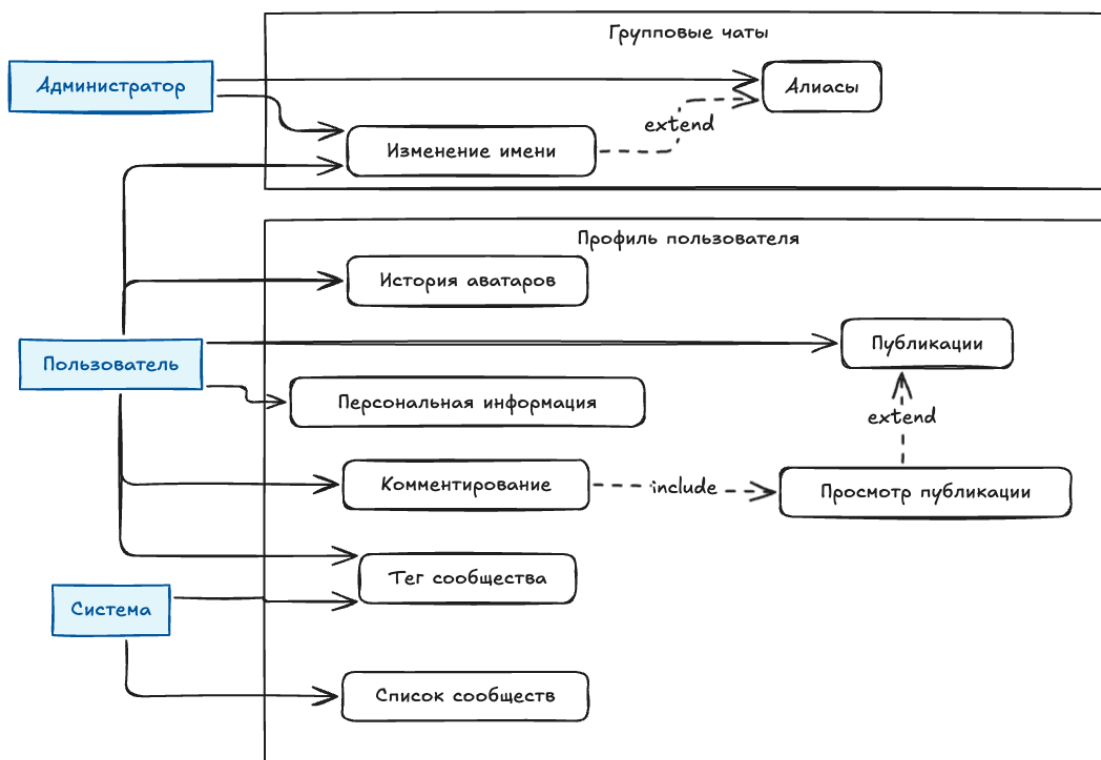


Рисунок 3 — Диаграмма прецедентов

2.1.2 Спецификация прецедентов

Основные прецеденты детализированы в соответствии с шаблоном RUP (Rational Unified Process) для формализации потоков событий.

UC1: Аутентификация пользователя

- **Идентификатор:** UC1
- **Участники (акторы):** Пользователь, Система, Внешний Matrix-сервер
- **Предусловия:** Пользователь имеет учетную запись на домашнем сервере Matrix и располагает действующим OpenID-токеном.
- **Постусловия:** Пользователь успешно аутентифицирован в системе msocial, создана локальная сессия в базе данных, клиенту возвращена пара JWT-токенов.
- **Основной поток:**
 1. Пользователь инициирует вход в систему через клиентское приложение.
 2. Клиент получает временный OpenID-токен от Matrix-сервера и отправляет его на бэкэнд msocial.

3. Бэкенд msocial отправляет запрос на верификацию токена в Matrix Federation API.

4. Matrix Federation API подтверждает валидность токена и возвращает идентификатор пользователя (Matrix ID).

5. Система выполняет поиск пользователя в локальной БД. Если пользователь отсутствует, запускается процесс провижининга (создания учетной записи).

6. Система генерирует пару JWT-токенов (access и refresh), сохраняет сессию в базе данных и возвращает токены клиенту.

- **Потоки исключений:**

- **Исключение 3а (Невалидный токен):** Matrix-сервер возвращает ошибку верификации. Система отклоняет запрос, возвращая код 401 Unauthorized.

- **Исключение 5а (Ошибка БД):** Локальная база данных временно недоступна. Система возвращает ошибку 500 Internal Server Error, данные сессии не сохраняются.

UC2: Управление профилем пользователя

- **Идентификатор:** UC2

- **Участники (акторы):** Пользователь, Система

- **Предусловия:** Пользователь успешно авторизован в системе (располагает валидным JWT-токеном).

- **Постусловия:** Глобальные данные профиля (отображаемое имя, статус) изменены и обновлены в локальной базе данных.

- **Основной поток:**

1. Пользователь открывает интерфейс настроек профиля.

2. Система запрашивает из БД текущие данные профиля и передает их на клиент.

3. Пользователь вносит изменения в поля формы (новое отображаемое имя, обновленный текстовый статус).

4. Пользователь нажимает кнопку «Сохранить».

5. Система выполняет валидацию введенных данных (проверка на запрещенные символы, длину строки).

6. Система обновляет запись пользователя в БД и возвращает успешный ответ.

- **Потоки исключений:**

– **Исключение 5а (Ошибка валидации):** Новое отображаемое имя превышает лимит в 100 символов. Система прерывает процесс, подсвечивает поле ввода и выводит предупреждение.

UC3: Управление публикациями

– **Идентификатор:** UC3

– **Участники (акторы):** Пользователь, Система

– **Предусловия:** Пользователь авторизован, его аккаунт не заблокирован модератором, размер вложений соответствует лимитам.

– **Постусловия:** Публикация создана, сохранена в БД и добавлена в ленту профиля.

– **Основной поток:**

1. Пользователь открывает редактор постов, вводит текст и загружает медиафайл.

2. Пользователь нажимает кнопку «Опубликовать».

3. Система проверяет права пользователя и лимиты на публикации (защита от спама).

4. Система валидирует размер и формат загруженного медиафайла.

5. Система сохраняет публикацию, присваивая ей уникальный ID, и возвращает созданный объект.

– **Потоки исключений:**

– **Исключение 4а (Неподдерживаемый формат медиа):** Пользователь загружает исполняемый файл вместо изображения. Система блокирует сохранение и возвращает ошибку 400 Bad Request.

UC4: Управление персональной информацией

– **Идентификатор:** UC4

– **Участники (акторы):** Пользователь, Система

– **Предусловия:** Пользователь авторизован, его профиль доступен для модификации.

– **Постусловия:** Личные данные (дата рождения, адрес, любимый трек) обновлены.

– **Основной поток:**

1. Пользователь открывает форму редактирования личных данных.

2. Система подгружает текущие значения полей `personal_infos`.

3. Пользователь заполняет поля и нажимает «Сохранить».

4. Система выполняет валидацию (формат даты, корректность URL-ссылки на трек).

5. Система перезаписывает данные в таблице `personal_infos` и уведомляет об успехе.

2.1.3 Глоссарий терминов

Основные термины определены на основе спецификации Matrix [2] и методологии предметно-ориентированного проектирования (DDD) [3].

Таблица 4 — Глоссарий терминов предметной области

Термин	Определение
Matrix ID (MXID)	Уникальный глобальный идентификатор пользователя в федеративной сети Matrix, имеющий вид <code>@username:domain.tld</code> .
OpenID Token	Краткосрочный токен безопасности, выдаваемый домашним сервером Matrix для подтверждения личности пользователя внешним службам.
Access Token	Краткосрочный JWT-токен доступа, используемый для авторизации каждого HTTP-запроса клиента к API <code>msocial</code> .
Refresh Token	Долгосрочный токен, хранящийся в базе данных и используемый клиентом для получения новой пары токенов после истечения <code>Access Token</code> .
Alias	Отображаемое имя (роль) пользователя, действующее исключительно в рамках конкретной группы/комнаты.
Community Tag	Текстовый пострикс (маркер), автоматически добавляемый к сообщениям и профилю пользователя при его активности от лица сообщества.
Soft Delete	Метод удаления записей в базе данных путем установки флага (например, <code>deleted_at</code>) без физического уничтожения строки.
PCMEF	Presentation-Control-Mediator-Entity-Foundation — паттерн многослойной архитектуры enterprise-приложений.
Domain Entity	Объект предметной области, обладающий уникальной идентичностью, сохраняющейся при изменении его атрибутов.
Value Object	Объект предметной области, не имеющий собственной идентичности и определяемый исключительно набором своих свойств (например, <code>PersonalInfo</code>).

Термин	Определение
Port	Спецификация интерфейса в гексагональной архитектуре, определяющая набор операций для взаимодействия домена с внешним миром.
Adapter	Технологическая реализация порта, связывающая бизнес-логику с конкретной технологией (базой данных, внешним API, файловым хранилищем).

2.2 Модель предметной области

2.2.1 Архитектура слоев приложения и модель классов проектирования

Логическая архитектура приложения построена с использованием многоуровневого подхода, характерного для корпоративных систем на платформе Spring. Она включает шесть функциональных слоёв:

1. **Controller Layer** — обрабатывает HTTP-запросы, выполняет валидацию входных DTO с помощью Jakarta Validation и преобразует их в доменные структуры.

2. **Service Layer** — содержит чистую бизнес-логику приложения, управляет границами транзакций и осуществляет координацию работы репозитория.

3. **Repository Layer** — обеспечивает интерфейсы для CRUD-операций и сложных запросов к базе данных, скрывая детали доступа к РСУБД.

4. **Entity Layer** — доменные сущности с аннотациями JPA, инкапсулирующие ключевые бизнес-правила и структуру таблиц.

5. **DTO Layer** — классы передачи данных (records), обеспечивающие безопасность и предотвращающие утечку внутренних деталей сущностей наружу.

6. **Exception Layer** — реализует централизованную обработку исключений с помощью @ControllerAdvice, преобразуя внутренние ошибки в понятные HTTP-статусы.

Основные сущности домена: User, PersonalInfo, Avatar, Post, Comment, Community, GroupMember.

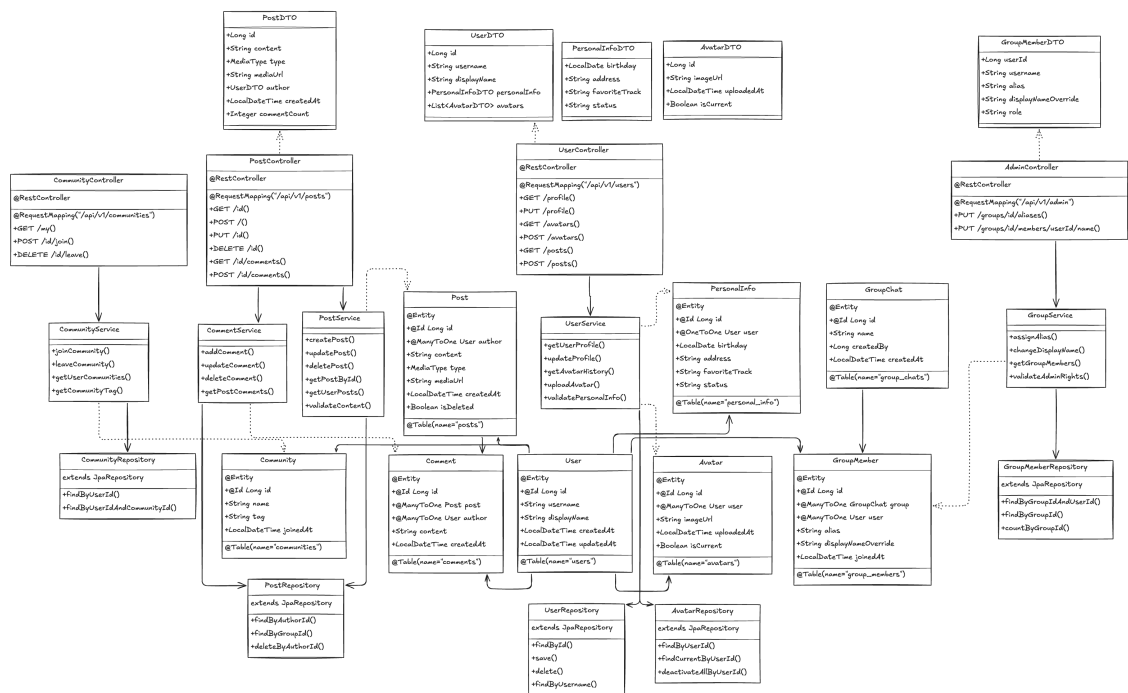


Рисунок 4 — Модель предметной области (Domain Model)

2.2.2 Описание сущностей и атрибутов

Распределение ответственности и привязка компонентов к технологическим слоям представлены в таблице ниже.

Таблица 5 — Описание слоёв и компонентов модели предметной области

Слой	Компоненты	Ответственность	Аннотации Spring/JPA
Controller	UserController, PostController, AuthController, CommentController, GroupController, CommunityController	Прием входящих запросов, валидация DTO, маппинг в доменные модели	@RestController, @RequestMapping, @PostMapping, @GetMapping, @Valid
Service	UserService, PostService, CommentService, GroupService, CommunityService, AuthService, SessionService	Оркестрация бизнес-процессов, управление транзакциями, вызовы портов	@Service, @Transactional
Repository	UserRepository, PostRepository, CommentRepository, GroupMemberRepository, CommunityRepository	Предоставление CRUD-интерфейсов, трансляция в SQL	@Repository, extends JpaRepository

Слой	Компоненты	Ответственность	Аннотации Spring/JPA
	CommunityRepository, SessionRepository		
Entity	User, PersonalInfo, Avatar, Post, Comment, Community, GroupMember, Session	Хранение состояния сущностей, ORM-отображение на таблицы БД	@Entity, @Table, @Id, @GeneratedValue, @OneToOne, @ManyToOne, @Version
DTO	UserDTO, PostDTO, CommentDTO, PersonalInfoDTO, AvatarDTO, LoginRequest, AuthResponse	Изоляция структуры API от изменений в доменных сущностях	Plain Java record classes, @Schema
Exception	GlobalExceptionHandler, NotFoundException, ValidationException, AccessDeniedException	Перехват исключений и формирование унифицированных ответов об ошибках	@ControllerAdvice, @ExceptionHandler, @ResponseStatus

2.2.3 Бизнес-правила

Бизнес-правила определяют инварианты системы — условия, которые должны оставаться истинными в любой момент времени при выполнении любых транзакций.

Таблица 6 — Бизнес-правила и ограничения предметной области

Правило	Описание	Последствия нарушения
BR-001	Один пользователь может иметь строго один активный аватар в текущий момент времени.	Нарушение целостности отображения профиля, путаница в клиенте.
BR-002	Назначение алиаса (роли) в группе разрешено только пользователям с правами администратора группы.	Нарушение политик безопасности, несанкционированное изменение ролей.
BR-003	Личная группа (приватное пространство) может быть создана в единственном	Дублирование приватных пространств, избыточное выделение ресурсов.

Правило	Описание	Последствия нарушения
	экземпляре для одного пользователя.	
BR-004	Тег сообщества отображается в профиле и сообщениях только для тех участников, статус которых активен.	Отображение недостоверной информации о членстве пользователя.
BR-005	Изменение отображаемого имени (алиаса) внутри группы не влечет за собой изменение глобального имени профиля.	Утрата глобальной идентичности пользователя в рамках сети.

Параграф описания логики правил:

- **Инвариант BR-001** гарантируется на уровне СУБД с помощью уникального частичного индекса `CREATE UNIQUE INDEX idx_current_avatar ON avatars(user_id) WHERE is_current = TRUE`. Это исключает возможность одновременного существования двух записей с флагом `is_current = true` для одного пользователя.

- **Правило BR-002** проверяется в слое бизнес-логики (GroupService) путем сопоставления прав запрашивающего пользователя с его статусом в таблице `group_members`.

- **Правило BR-005** реализует разделение областей видимости: глобальный профиль хранится в `users` и `personal_infos`, в то время как локальные переопределения имен участников чатов пишутся в таблицу `group_members`.

2.3 Архитектурное проектирование

2.3.1 Выбор архитектурного стиля

Для проектирования системы был выбран гибридный подход, сочетающий в себе классический пятислойный паттерн многоуровневой архитектуры **PCMEF** (Presentation-Control-Mediator-Entity-Foundation) [4] & принципы **гексагональной архитектуры** (Ports & Adapters) с функционально-ориентированной декомпозицией (feature-first).

Обоснование выбора данных паттернов:

1. **Четкое разделение ответственности:** PCMEF разделяет представление данных (Presentation), обработку HTTP-протокола (Control), логику бизнес-процессов (Mediator), доменные модели (Entity) и доступ к постоянному хранилищу (Foundation).

2. **Изоляция домена (Hexagonal Architecture):** бизнес-логика (Mediator) не зависит от используемой СУБД, библиотек сериализации и внешних протоколов. Домен взаимодействует с внешним миром через порты (интерфейсы), а инфраструктурные адаптеры реализуют эти порты. Это позволяет легко заменить СУБД PostgreSQL на NoSQL-решение или S3-хранилище без изменения кода бизнес-логики.

3. **Модульность (Feature-First):** в отличие от традиционного слоистого подхода (layer-first), где пакеты группируются по техническому признаку (все контроллеры в одном пакете, все сервисы в другом), система делится на доменные фичи (user, post, comment, auth, group). Это упрощает масштабирование проекта, повышает читаемость кода и упрощает последующий переход к микросервисной архитектуре.

2.3.2 Диаграмма пакетов (PCMEF)

Внутренняя структура пакетов проекта msocial организована следующим образом:

```
com.app.backendapi
├─ feature/
│   ├─ user/      (dto, controller, service, entity, repository)
│   ├─ post/      (аналогично)
│   ├─ comment/   (аналогично)
│   ├─ group/     (аналогично)
│   └─ auth/      (dto, controller, service, entity, repository,
port, adapter)
├─ common/
│   ├─ exceptions/ (ValidationException, NotFoundException)
│   ├─ security/   (AuthFilter, JwtProvider, SecurityConfig)
│   ├─ config/     (WebMvcConfig, CaffeineCacheConfig)
│   └─ utils/      (SecureTokenGenerator, ValidatorUtil)
```

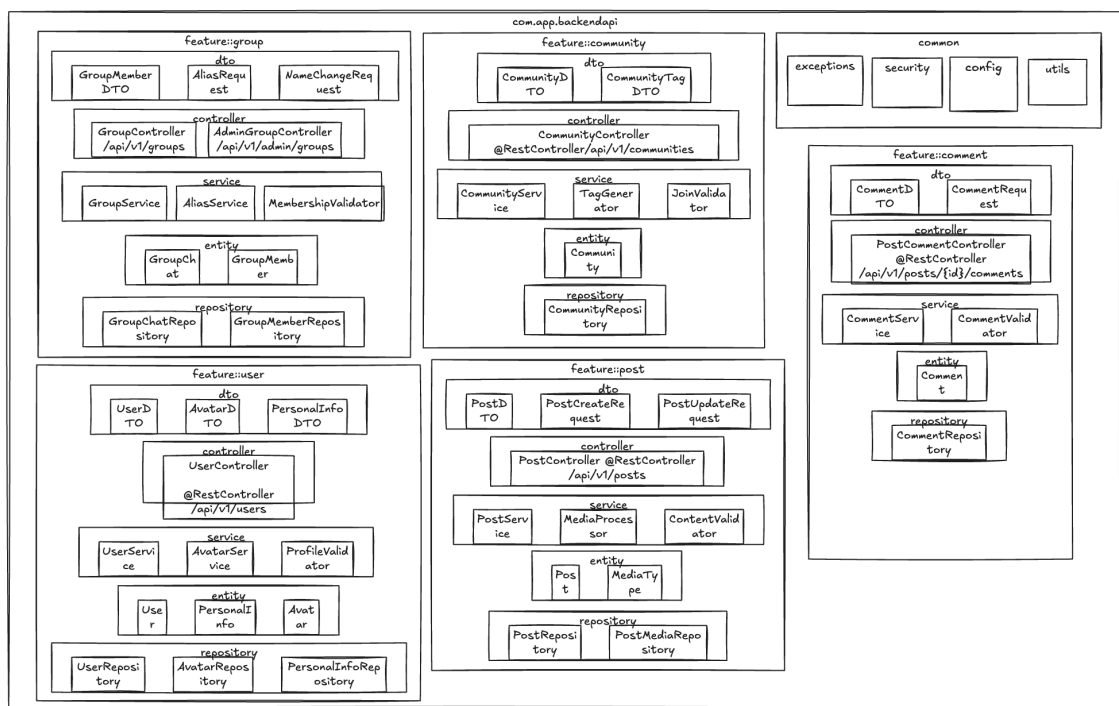



Рисунок 5 — Диаграмма пакетов PCMEF backend-слоя

2.3.3 Описание слоёв и их ответственности

Распределение ролей в рамках PCMEF-архитектуры приведено в таблице ниже.

Таблица 7 — Распределение компонентов по слоям PCMEF

Слой PCMEF	Компоненты проекта	Назначение
Presentation	UserDTO, PostDTO, CommentDTO, PersonalInfoDTO, PostCreateRequest, CommentRequest	Определение структур данных, используемых в API; сериализация и десериализация; Swagger-документация.
Control	UserController, PostController, CommentController, GroupController, AuthController	Обработка HTTP-запросов, проверка прав доступа на уровне ролей, валидация входных полей, вызов портов Mediator.
Mediator	UserService, PostService, CommentService, GroupService, AuthService, ProfileValidator, ContentValidator	Реализация сценариев использования (Use Cases), управление распределенными транзакциями, оркестрация бизнес-процессов.

Слой PCMEF	Компоненты проекта	Назначение
Entity	User, PersonalInfo, Avatar, Post, Comment, Community, GroupMember, Session	Представление доменных сущностей, инкапсуляция инвариантов бизнес-логики, JPA-маппинг.
Foundation	UserRepository, PostRepository, CommentRepository, GroupMemberRepository, SessionRepository, CaffeineCache	Реализация доступа к физической базе данных, кэширование, внешние вызовы (Matrix API).

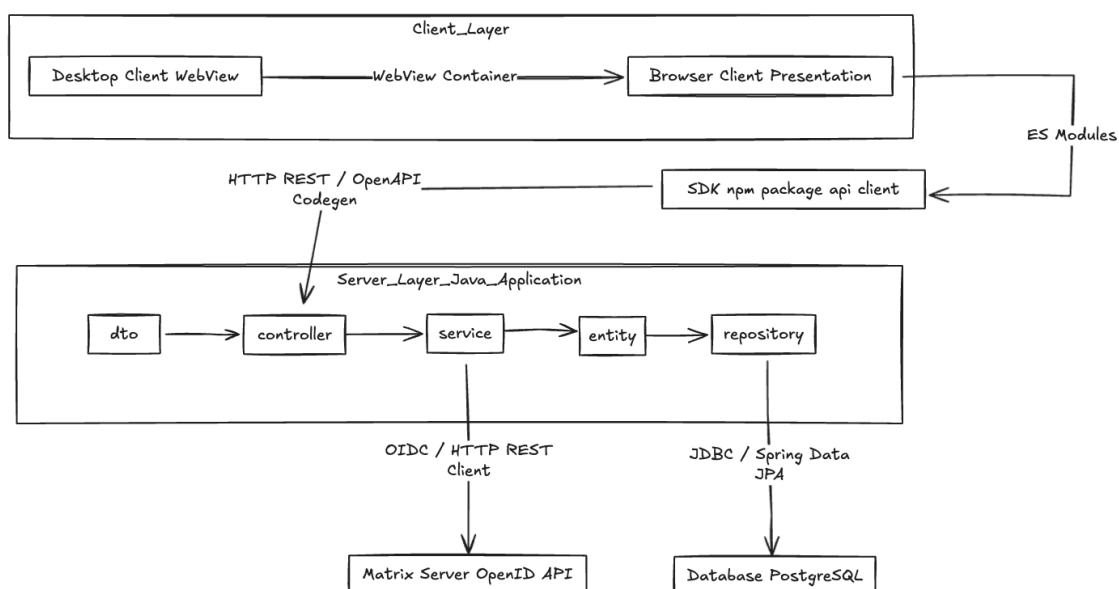


Рисунок 6 — Диаграмма инфраструктуры и развёртывания

Архитектурные решения задокументированы в формате ADR (Architecture Decision Records). Основные из них:

- **ADR-001 (Выбор PCMEF + Feature-First):** обеспечивает независимое тестирование и изолированное ведение разработки по фичам.
- **ADR-002 (Выбор PostgreSQL и JPA):** реляционная СУБД PostgreSQL [5] выбрана из-за строгой связности данных и требований к транзакциям (ACID).
- **ADR-003 (Аутентификация через Matrix OpenID):** исключает хранение паролей на стороне msocial, делегируя идентификацию домашнему серверу Matrix.

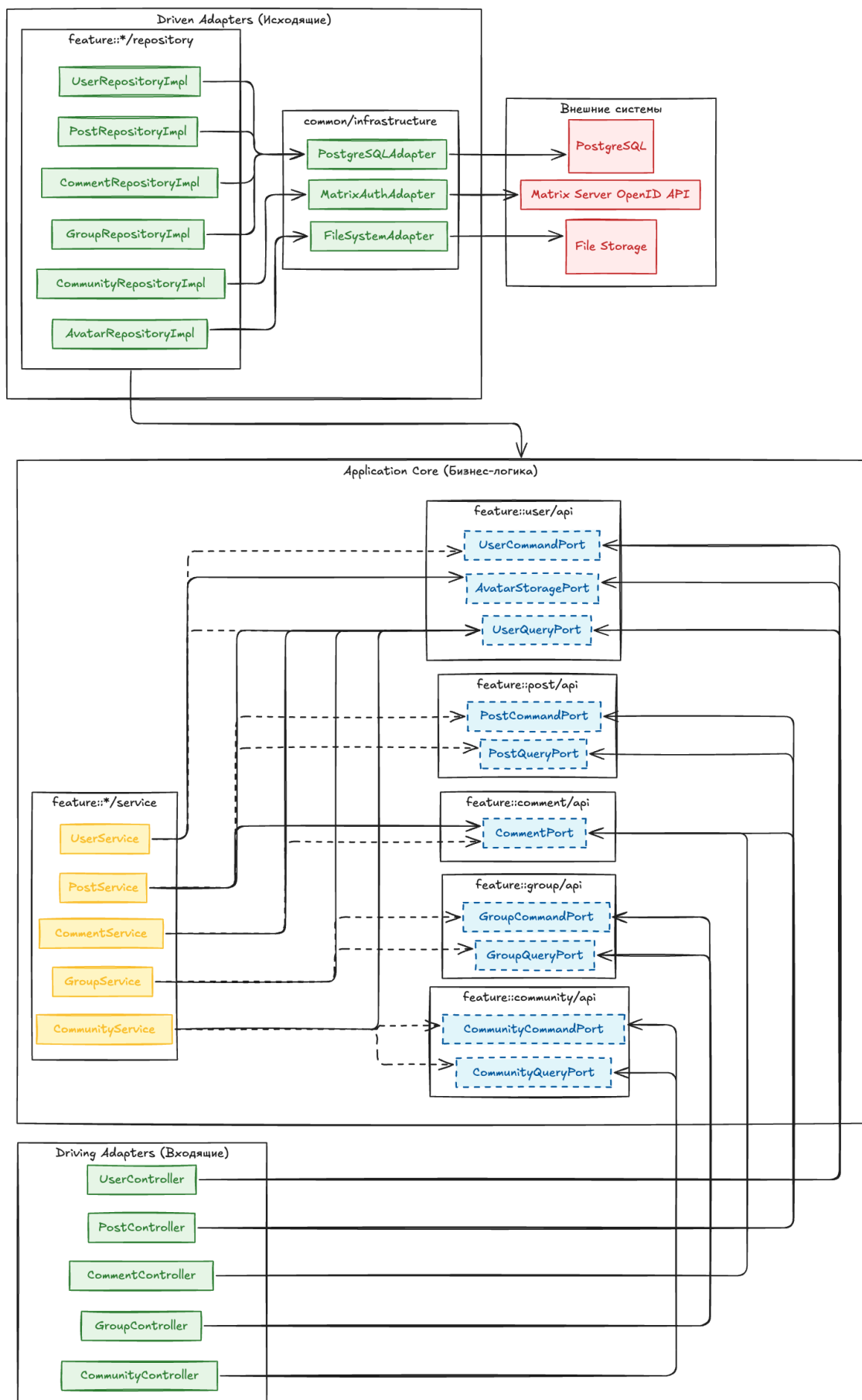


Рисунок 7 — Диаграмма интерфейсов (Ports and Adapters)

2.4 Проектирование базы данных

2.4.1 ER-диаграмма

Логическая схема базы данных разработана с учетом обеспечения целостности данных и высокой скорости выполнения запросов на чтение. База содержит следующие таблицы:

- `users` — учетные записи пользователей с глобальными именами.
- `personal_infos` — детальная личная информация (связь 1:1 с `users`).
- `avatars` — история загруженных аватарок (связь 1:N с `users`).
- `communities` — сообщества.
- `group_members` — таблица членства пользователей в сообществах с поддержкой локальных алиасов (ролей).
- `posts` — публикации пользователей (связь 1:N с `users`).
- `comments` — древовидные комментарии к постам (связь 1:N с `posts` и иерархическая связь `parent_id` 1:N к самой себе).
- `auth_sessions` — активные пользовательские сессии (связь 1:N с `users`).

2.4.2 Физическая модель данных и нормализация (1НФ, 2НФ, 3НФ)

Нормализация схемы базы данных выполнена последовательно до третьей нормальной формы (3НФ):

1. **Первая нормальная форма (1НФ):** все значения в ячейках таблиц являются атомарными. Списки аватарок вынесены из таблицы пользователей в отдельную таблицу `avatars`. Поля типа дат приведены к строгим типам `DATE` и `TIMESTAMP WITH TIME ZONE`.

2. **Вторая нормальная форма (2НФ):** схема находится в 1НФ, и каждый неключевой атрибут функционально полно зависит от первичного ключа. Все таблицы имеют простые суррогатные первичные ключи `id` (автоинкрементные суррогатные ключи `SERIAL`).

3. **Третья нормальная форма (3НФ):** схема находится во 2НФ, и неключевые атрибуты не имеют транзитивных зависимостей от первичных ключей.

– Личная информация пользователя вынесена в отдельную таблицу `personal_infos`. В исходной модели поля `address` и `birthday` транзитивно зависели от пользователя, что порождало риск аномалий при частом обновлении. Теперь `personal_infos` связана связью 1:1 с `users` через уникальное поле `user_id`.

- В таблице `group_members` добавлено уникальное композитное ограничение `UNIQUE(group_id, user_id)` для исключения дублирования записей членства.

- Для сущности `Avatar` добавлено ограничение целостности: в таблице `avatars` разрешен только один активный аватар для каждого пользователя. Это реализовано с помощью частичного уникального индекса: `CREATE UNIQUE INDEX idx_current_avatar ON avatars(user_id) WHERE is_current = TRUE;`

2.4.3 DDL-скрипты

Скрипты создания структуры таблиц приведены в Приложении Б.

2.5 Детальное проектирование

2.5.1 Диаграммы последовательности

Для детального описания взаимодействия объектов в системе спроектированы три основных сценария:

1. **UC4: Сохранение профиля (Управление персональной информацией).** Сценарий описывает процесс изменения личных данных пользователя.

- Контроллер принимает HTTP PUT запрос, валидирует DTO.
- Вызывается метод `updateProfile` в сервисе `UserService`.
- Сервис проверяет существование пользователя через `UserQueryPort`.
- Сервис обращается к `PersonalInfoRepository` для обновления данных в таблице `personal_infos`.

- Транзакция завершается, и обновленные данные кэшируются.

2. **UC3: Создание публикации.** Описывает процесс добавления поста.

- Клиент отправляет POST запрос с текстом публикации.
- `PostController` проверяет валидность текста.
- `PostService` через `ContentValidator` проверяет текст на спам и запрещенные слова (паттерн `Chain of Responsibility`).
- При успешной проверке пост сохраняется в БД через `PostRepository`.

3. **UC3-2a: Обработка и валидация медиафайлов.** Описывает циклическую обработку вложений перед публикацией.

- При добавлении файлов в пост, сервис `MediaProcessor` циклически опрашивает каждый файл.

- Проверяется MIME-тип и размер файла. При превышении лимитов генерируется исключение `ValidationException`.

- Валидные файлы загружаются в хранилище через порт `MediaStoragePort`.

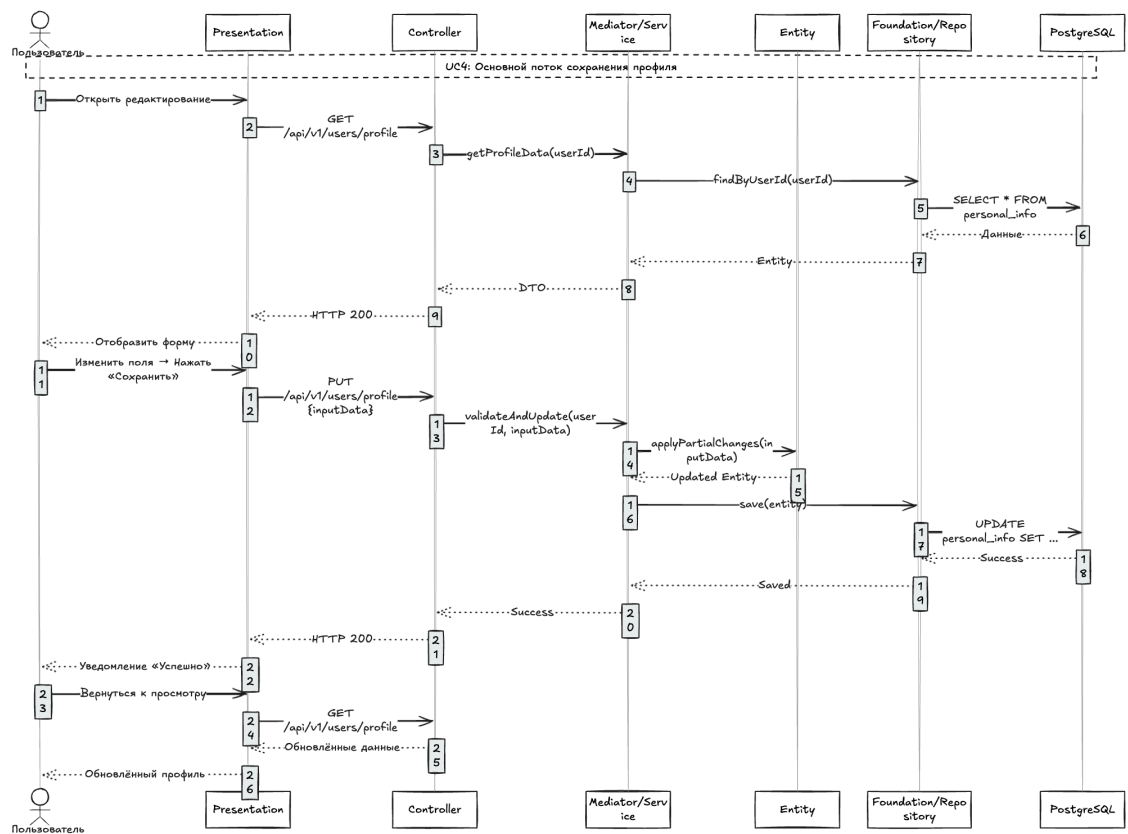


Рисунок 8 — Диаграмма последовательности UC4: основной поток сохранения профиля

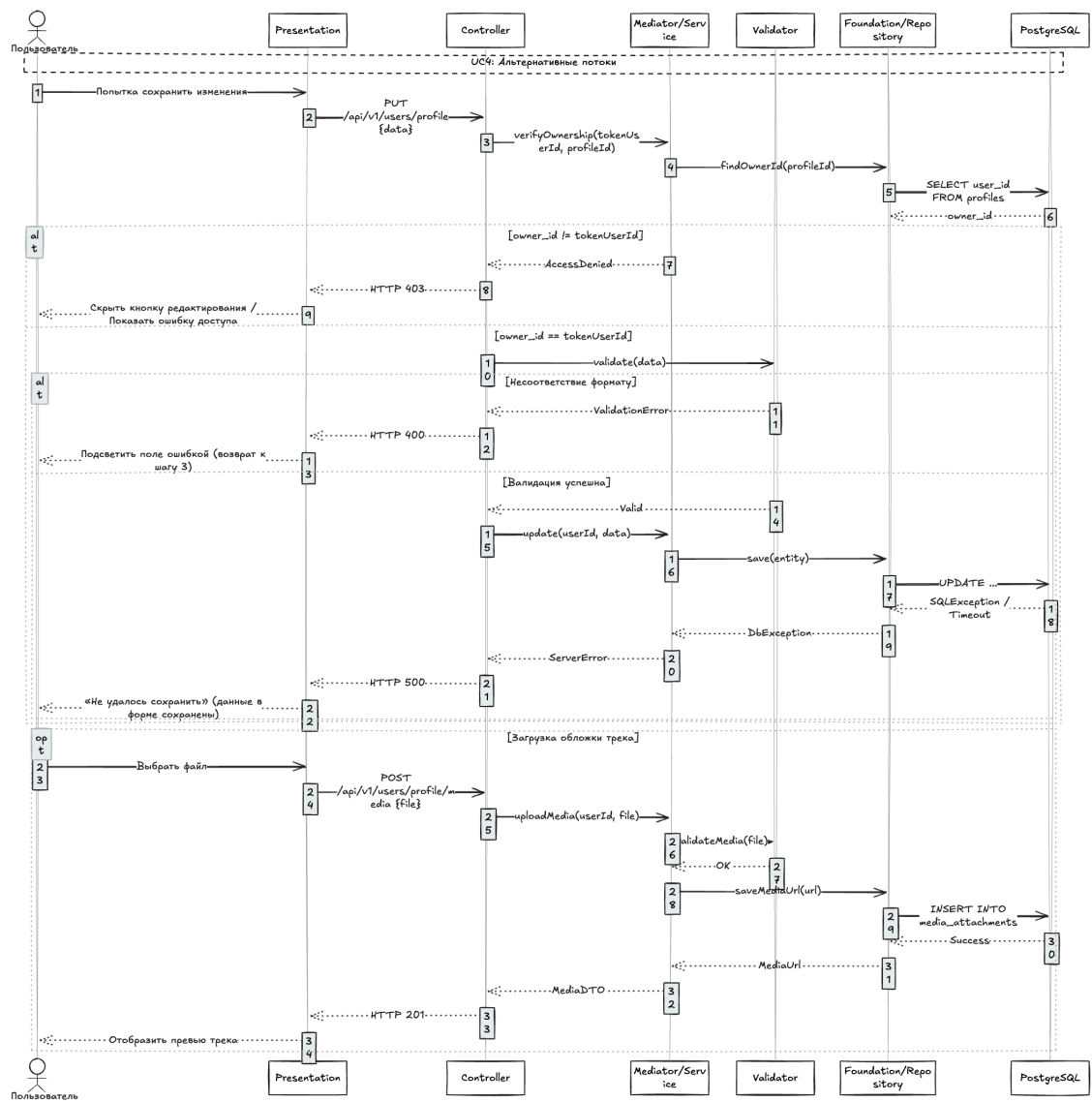


Рисунок 9 — Диаграмма последовательности UC4: альтернативные потоки

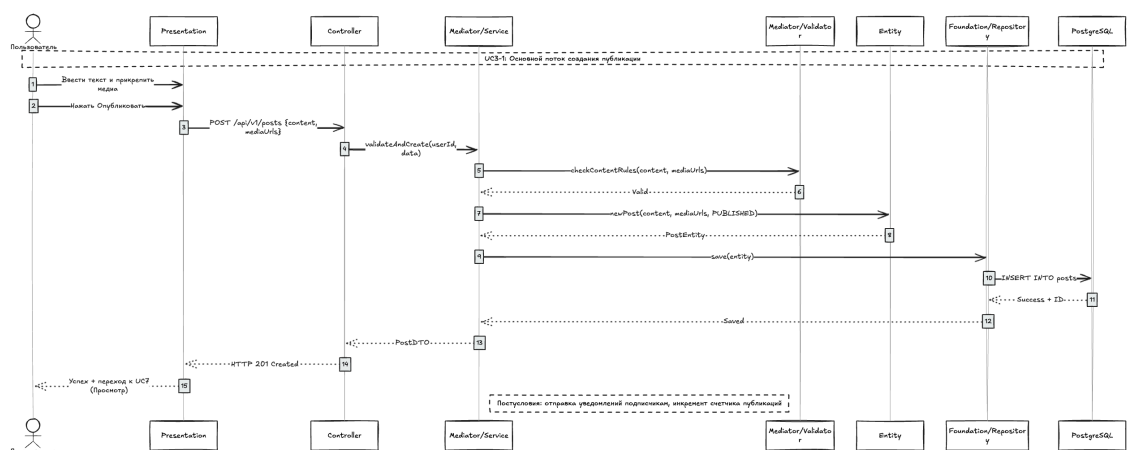


Рисунок 10 — Диаграмма последовательности UC3-1: создание публикации

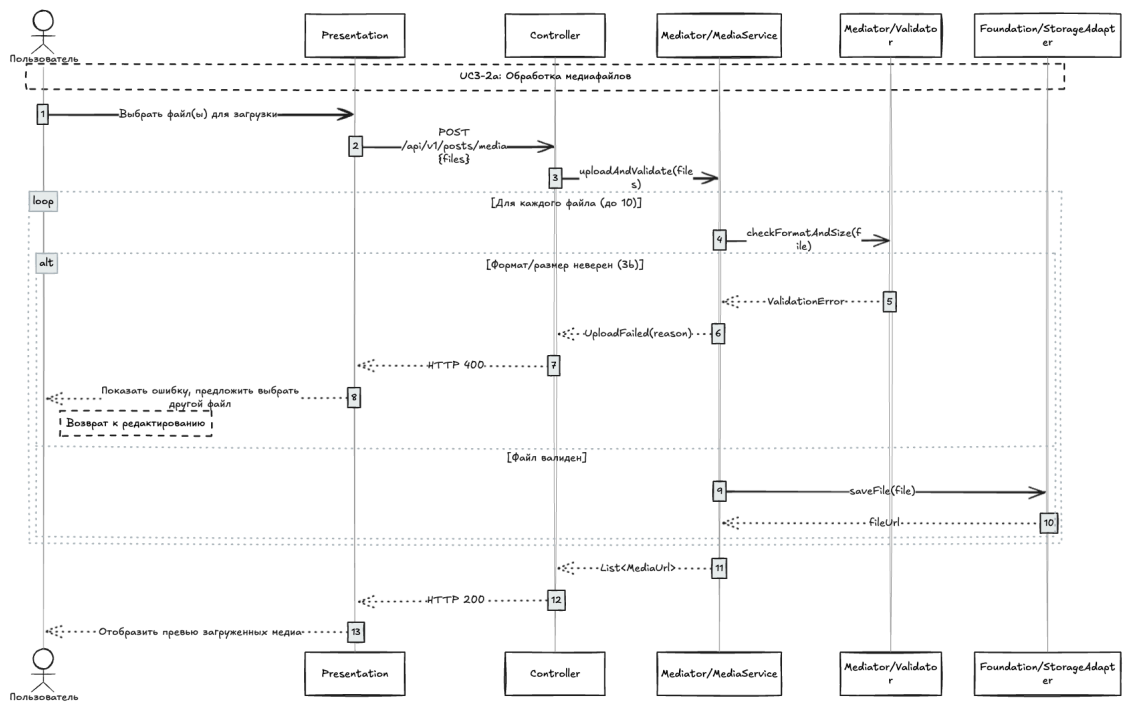


Рисунок 11 — Диаграмма последовательности UC3-2a: обработка медиафайлов

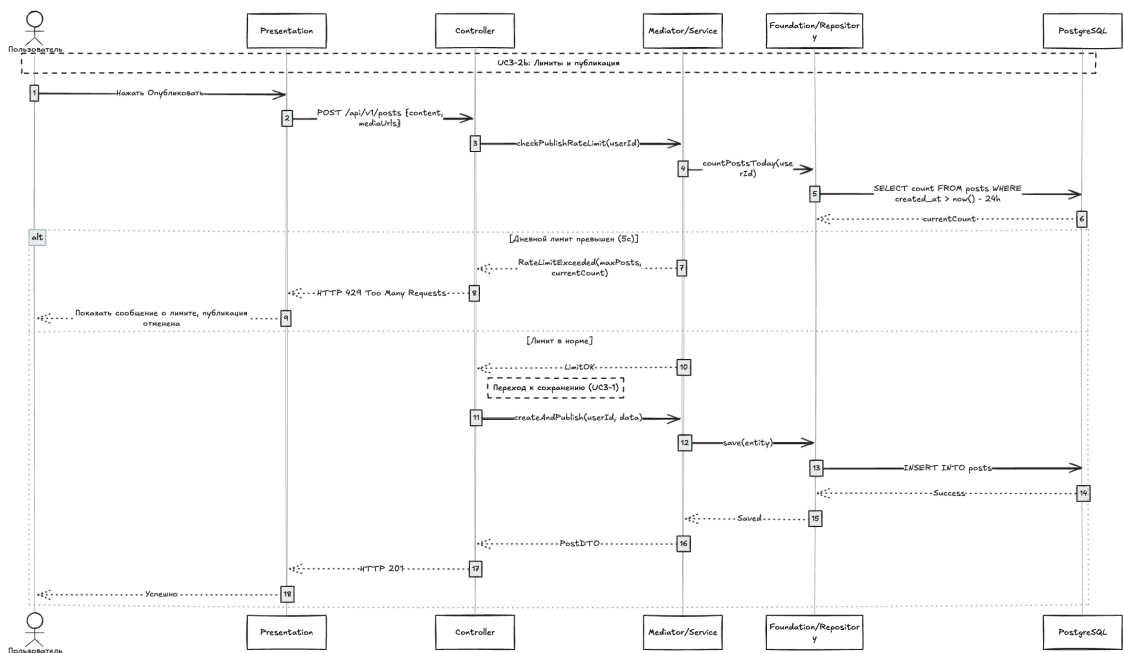


Рисунок 12 — Диаграмма последовательности UC3-2b: проверка лимитов

2.5.2 Диаграммы классов проектирования

Модель классов проектирования организована в соответствии с функциональными фичами проекта:

- **feature::user**: содержит класс сущности User, связанную сущность PersonalInfo и Avatar. Логика сосредоточена в UserService и AvatarService.

- **feature::auth**: содержит сущность `Session`, сервис `AuthService`, управляющий жизненным циклом сессий, генератор токенов `TokenService` и адаптеры для интеграции с Matrix OpenID API.
- **feature::post**: содержит сущности `Post` и перечисление `MediaType`. Доступ к БД осуществляется через `PostRepository`, логика — в `PostService`.
- **feature::comment**: содержит сущность `Comment` с древовидной ссылкой на саму себя (`parent`).

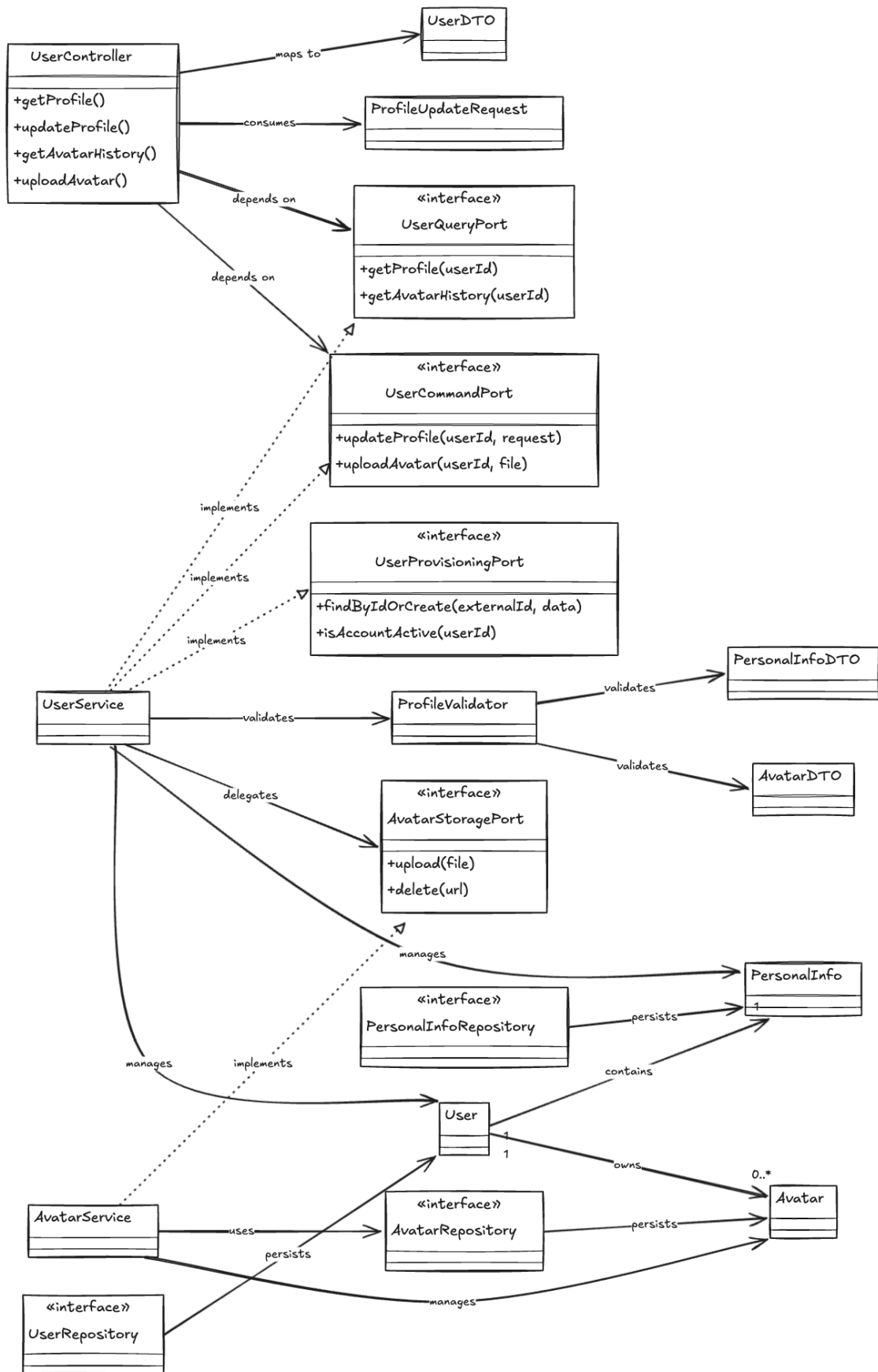


Рисунок 13 — Диаграмма классов: управление пользователями

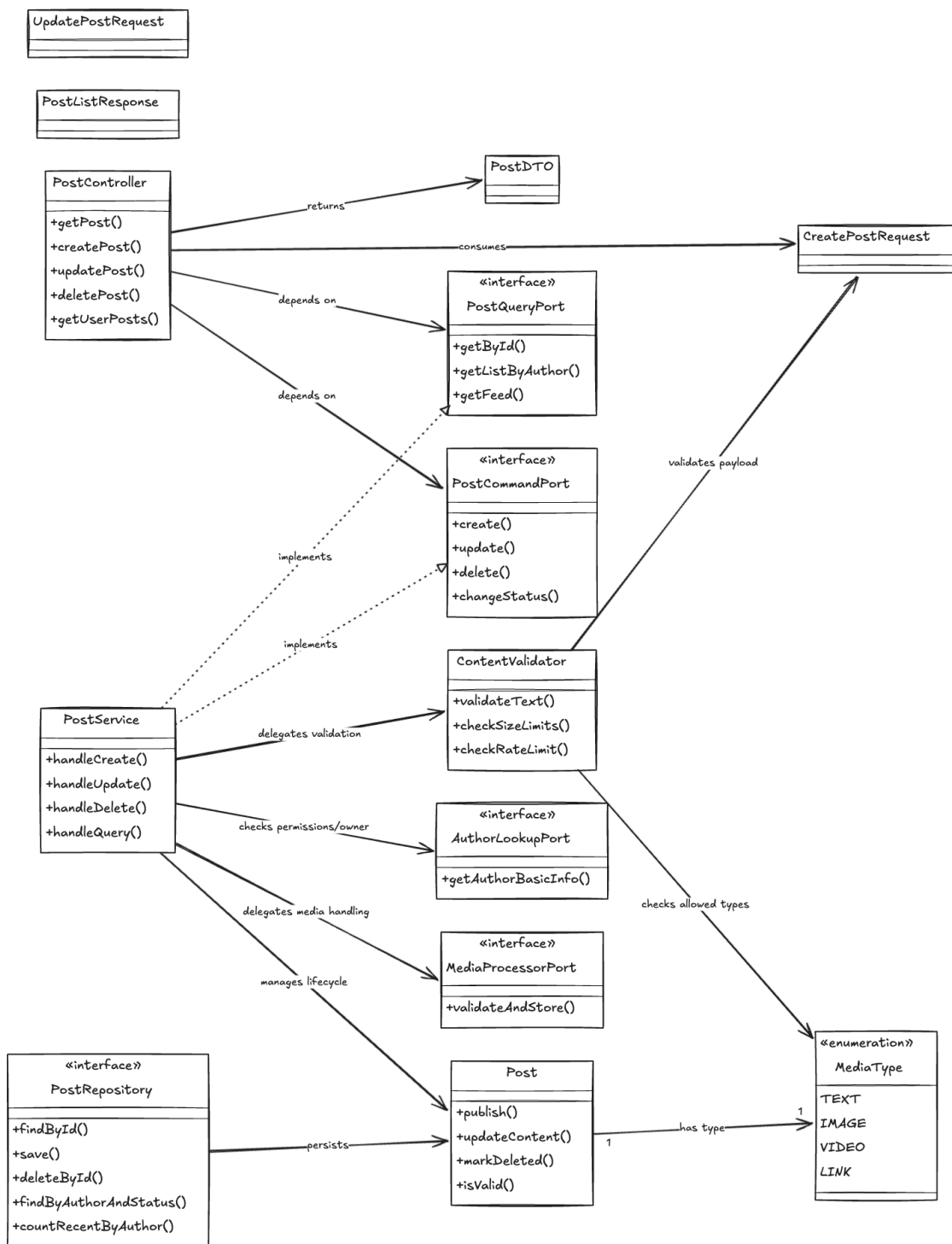


Рисунок 15 — Диаграмма классов: публикации

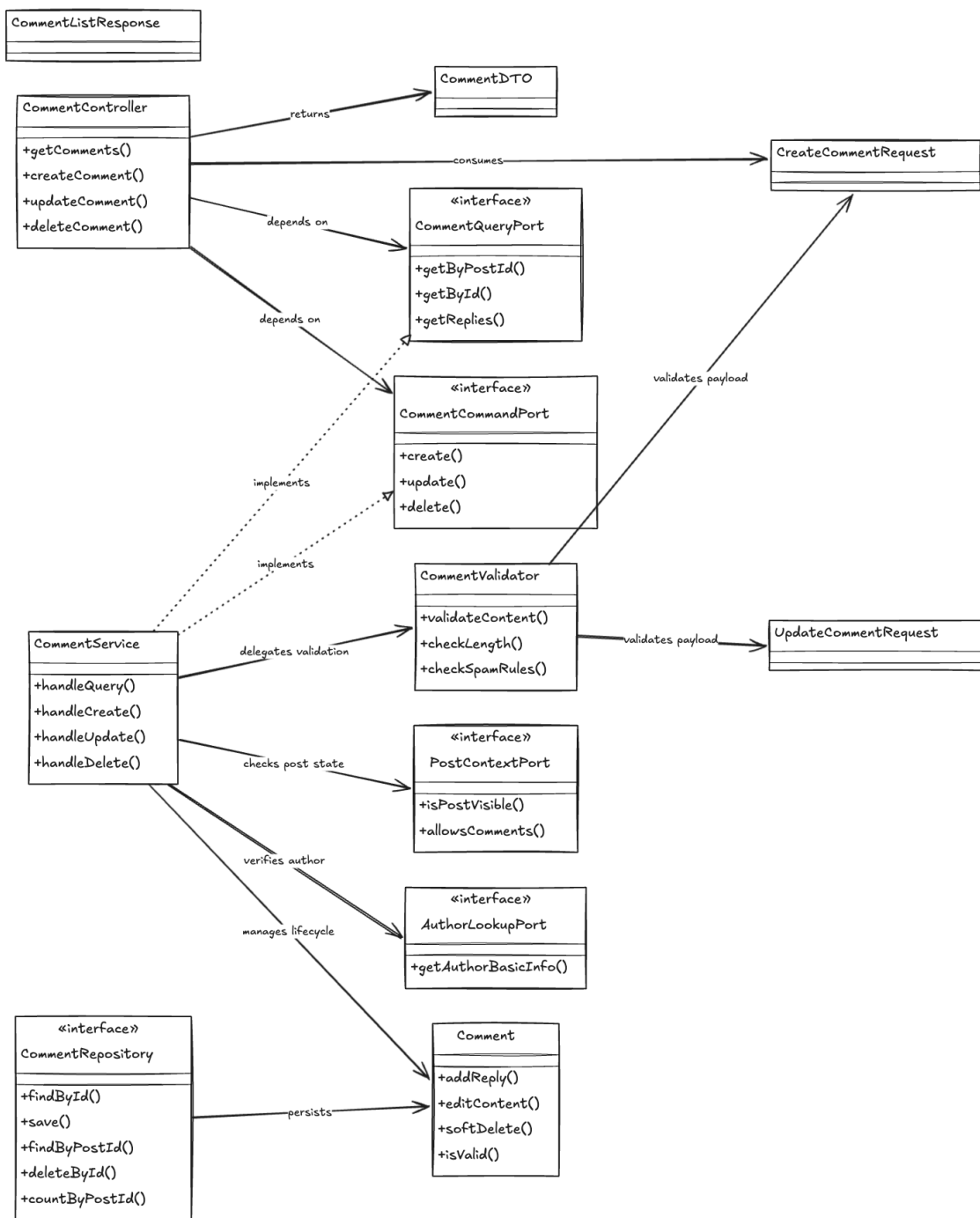


Рисунок 16 — Диаграмма классов: комментарии

2.5.3 Применение паттернов GoF

При детальном проектировании системы были применены классические паттерны банды четырех (GoF) [6]:

1. **Builder**: используется для создания сложных DTO и доменных сущностей с множеством опциональных полей (например, Session, User, Post). Реализован с помощью аннотации @Builder библиотеки Lombok. Это избавляет от необходимости писать громоздкие конструкторы и повышает читаемость кода:

```
Session session = Session.builder()  
    .userId(userId)  
    .refreshToken(token)  
    .refreshTokenExpiresAt(expiry)  
    .createdAt(Instant.now())  
    .build();
```

2. **Decorator (Декоратор)**: применяется для прозрачного кэширования и сбора метрик для портов без изменения их исходных реализаций. Например, класс `CachedUserQueryPort` является декоратором для базового репозитория `UserRepository`. При вызове метода получения пользователя декоратор сначала обращается к Caffeine-кэшу, и только при промахе запрашивает базу данных.

3. **Chain of Responsibility (Цепочка обязанностей)**: используется в подсистеме модерации публикаций и комментариев. Текст публикации проходит через цепочку независимых обработчиков-валидаторов:

`LengthValidator` (проверка длины)

└─ `SpamValidator` (проверка на спам по ключевым словам)

└─ `ToxicityValidator` (анализ токсичности)

Каждый элемент цепочки проверяет текст и либо передает его следующему обработчику, либо прерывает обработку, выбрасывая `ValidationException`.

4. **Adapter (Адаптер)**: используется для интеграции с внешними системами. Класс `MatrixFederationAdapter` адаптирует HTTP-интерфейс `Matrix Federation API` под доменный порт `MatrixAuthPort`, выполняя преобразование внешних ответов сервера во внутренние структуры данных. Класс `MediaStorageAdapter` адаптирует вызовы API S3-совместимого объектного хранилища (`MinIO` / `AWS S3`) к доменному порту `AvatarStoragePort`.

3 Реализационная часть

3.1 Реализация бизнес-логики

Бэкенд-система msocial реализована на языке Java 21 с использованием фреймворка Spring Boot 3.2 [7]. В соответствии с выбранной архитектурой РСМЕФ и принципами Clean Architecture, бизнес-логика изолирована от внешних фреймворков и библиотек.

3.1.1 Структура проекта

Проектная структура следует шаблону **feature-first**, где код сгруппирован по функциональным модулям:

```
msocial/
├─ src/main/java/lghdnov/msocial/
│   └─ feature/
│       ├── user/          (dto, controller, service, entity,
repository, port, adapter)
│       ├── auth/          (аналогично)
│       ├── post/          (аналогично)
│       └─ comment/        (аналогично)
│   └─ common/
│       ├── exceptions/    (ValidationException,
GlobalExceptionHandler)
│       ├── security/      (AuthFilter, JwtProvider, SecurityConfig)
│       ├── config/        (WebMvcConfig, CaffeineCacheConfig)
│       └─ utils/          (SecureTokenGenerator)
```

3.1.2 Классы-сущности (Entity Layer)

Ниже представлены листинги ключевых JPA-сущностей, инкапсулирующих структуру базы данных и доменное поведение.

Листинг 3.1 — Класс доменной сущности User

```
@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "username", nullable = false, unique = true,
length = 50)
    private String username;

    @Column(name = "display_name", nullable = false, length = 100)
    private String displayName;
```

```

@Column(name = "created_at", nullable = false, updatable = false)
private Instant createdAt;

@Column(name = "last_active")
private Instant lastActive;

@OneToOne(mappedBy = "user", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
private PersonalInfo personalInfo;

@OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
orphanRemoval = true)
@OrderBy("uploadedAt DESC")
private List<Avatar> avatars = new ArrayList<>();
}

```

Листинг 3.2 — Сущность PersonalInfo

```

@Entity
@Table(name = "personal_infos")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class PersonalInfo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "birthday")
    private LocalDate birthday;

    @Column(name = "address", length = 255)
    private String address;

    @Column(name = "favorite_track_url", length = 512)
    private String favoriteTrackUrl;

    @Column(name = "status", length = 255)
    private String status;
}

```

Листинг 3.3 — Сущность Session


```

@Entity
@Table(name = "auth_sessions")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Session {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", nullable = false)
    private Long userId;

    @Column(name = "refresh_token", unique = true, length = 512)
    private String refreshToken;

    @Column(name = "refresh_token_expires_at", nullable = false)
    private Instant refreshTokenExpiresAt;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    @Column(name = "revoked_at")
    private Instant revokedAt;

    @Version
    private Long version;

    public boolean isActive() {
        return revokedAt == null &&
refreshTokenExpiresAt.isAfter(Instant.now());
    }
}

```

3.1.3 Слой доступа к данным (Foundation Layer)

Доступ к данным реализован с помощью Spring Data JPA. Пример репозитория управления сессиями:

Листинг 3.4 — Интерфейс SessionRepository

```

@Repository
public interface SessionRepository extends JpaRepository<Session,
Long> {
    Optional<Session> findByRefreshToken(String refreshToken);

    @Modifying
    @Query("UPDATE Session s SET s.revokedAt = CURRENT_TIMESTAMP

```

```

WHERE s.id = :id")
    void revokeSession(@Param("id") Long id);

    List<Session> findAllByUserIdAndRevokedAtIsNull(Long userId);
}

```

3.1.4 Слой бизнес-логики (Mediator Layer)

Сервисы реализуют доменные порты, обеспечивая слабую связанность. Пример реализации процесса аутентификации:

Листинг 3.5 — Метод login в AuthService

```

@Service
@RequiredArgsConstructor
public class AuthService implements AuthCommandPort {
    private final MatrixAuthPort matrixAuthPort;
    private final UserCommandPort userCommandPort;
    private final SessionManagementPort sessionManagementPort;
    private final TokenGenerationPort tokenGenerationPort;

    @Override
    @Transactional
    public AuthResponse login(LoginRequest request) {
        // 1. Верификация во внешней сети Matrix
        String matrixId =
matrixAuthPort.verifyOpenIdToken(request.openidToken());

        // 2. Поиск или создание локального пользователя
        (provisioning)
        User user = userCommandPort.findByUsername(matrixId)
            .orElseGet(() -> userCommandPort.create(User.builder()
                .username(matrixId)
                .displayName(extractLocalName(matrixId))
                .createdAt(Instant.now())
                .build()));

        // 3. Создание сессии и генерация токенов
        Session session =
sessionManagementPort.createSession(user.getId());
        String accessToken =
tokenGenerationPort.generateAccessToken(user, session.getId());

        return new AuthResponse(accessToken,
session.getRefreshToken(), 900L);
    }
}

```

```

private String extractLocalName(String mxid) {
    return mxid.substring(1, mxid.indexOf(":"));
}
}

```

3.2 Рефакторинг и оптимизация

3.2.1 Статический анализ кода

Для контроля качества исходного кода в Gradle-сборку внедрены плагины статического анализа SpotBugs, PMD и Checkstyle. Внедрен плагин JaCoCo для замера покрытия кода автоматизированными тестами.

- Покрытие критических сервисных классов (сервисы модулей auth и user) составляет не менее 75%.

- Цикломатическая сложность методов строго ограничена: максимальный индекс сложности не превышает 8, что гарантирует простоту чтения и поддержки кода.

- Проверка зависимостей на уязвимости осуществляется плагином OWASP Dependency Check при каждой сборке CI/CD пайплайна по базе уязвимостей [8].

3.2.2 Применение паттернов (Data Mapper, Identity Map)

- **Data Mapper**: преобразование сущностей в DTO осуществляется с помощью MapStruct. Это изолирует доменную структуру БД от внешнего API, предотвращая утечку полей (например, revokedAt, version).

- **Identity Map**: повторные запросы на чтение данных пользователя в рамках одного HTTP-запроса оптимизируются с помощью кэша первого уровня Hibernate. На уровне приложения реализован кэш второго уровня на базе библиотеки Caffeine с TTL 5 минут для статических данных профиля.

3.2.3 Оптимизация SQL-запросов

Для повышения пропускной способности СУБД выполнены следующие настройки:

- Установлен ленивый тип загрузки связей (FetchType.LAZY) для всех ассоциаций @OneToMany и @ManyToOne для исключения проблемы N+1.

- Созданы составные индексы в PostgreSQL:

```

CREATE INDEX idx_posts_author_created ON posts(author_id, created_at
DESC);

```

Это ускоряет выборку новостной ленты конкретного пользователя с сортировкой по дате.

– Добавлена пагинация на уровне базы данных с использованием объектов Pageable и Slice в Spring Data, что предотвращает перегрузку ОЗУ при обработке миллионов комментариев.

3.3 Пользовательский интерфейс

3.3.1 Настольное (desktopное) приложение

Настольный клиент представляет собой легковесную оболочку, созданную на базе фреймворка Tauri. Логика отрисовки интерфейса выполняется внутри WebView, а взаимодействие с операционной системой и сервером msocial осуществляется по защищенному протоколу HTTPS через REST API.

3.3.2 Веб-приложение

Клиентская часть likma разработана на React 18 [9], сборщике Vite и компиляторе TypeScript.

– **CSS-in-JS**: стилизация выполнена с использованием vanilla-extract, что гарантирует компиляцию стилей на этапе сборки (zero-runtime CSS) и высокую скорость рендеринга.

– **Управление состоянием**: Jotai обеспечивает реактивное атомарное состояние сессии и настроек на клиенте.

Пример Jotai-атома авторизации:

Листинг 3.6 — Атом Jotai для сессии пользователя

```
import { atom } from 'jotai';

export interface AuthSession {
  accessToken: string;
  refreshToken: string;
  userId: string;
}

export const sessionAtom = atom<AuthSession | null>(null);
export const isAuthenticatedAtom = atom((get) => get(sessionAtom) !== null);
```

– **TanStack Query (React Query)**: используется для кэширования ответов сервера, автоматического обновления данных в фоне (background refetching) и оптимистичных обновлений интерфейса при лайках и комментариях.

3.3.3 Сравнение реализаций

Таблица 8 — Сравнение реализаций web и desktop приложений

Критерий	Web (SPA)	Desktop (Tauri)
Доставка пользователю	Через браузер по URL	Локальный установочный пакет
Потребление памяти	Зависит от открытых вкладок браузера	Минимальное (WebView от ОС, 40 МБ)
Локальные возможности	Ограничены песочницей браузера	Прямой доступ к ФС через Rust-биндинги
Автономность (Offline)	Service Workers, кэширование	Локальная база данных SQLite + синхронизация

3.4 Безопасность и транзакции

3.4.1 Аутентификация и авторизация

Схема безопасности базируется на Spring Security. Конфигурация включает в себя пользовательский фильтр аутентификации JWT с интеграцией через Matrix OpenID Federation [10] и генерацией внутренней JWT-пары по стандарту RFC 7519 [11]:

Листинг 3.7 — Конфигурационный класс SecurityConfig

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(cors ->
                cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/login", "/api/v1/auth/refresh")
                .permitAll()
                .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/scalar/**")
                .permitAll()
                .anyRequest().authenticated()
            )
    }
}
```

```

        .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

3.4.2 Управление транзакциями

Управление транзакциями в Spring Boot настроено декларативно с помощью аннотации `@Transactional` [12]. Все методы на чтение оптимизированы с помощью `@Transactional(readOnly = true)`. Это сообщает Hibernate о возможности отключения механизма dirty checking, что существенно экономит ресурсы процессора.

3.4.3 Защита от атак и Rate Limiting

- **CORS**: настроен жесткий фильтр источников. Разрешены запросы только от доверенного домена веб-приложения, настроен проброс заголовков `Authorization` и `Set-Cookie`.

- **SQL-инъекции**: все репозитории используют именованные параметры и компилируемые `PreparedStatement`, что делает SQL-инъекции невозможными.

- **Rate Limiting (Bucket4j)**: для защиты API от перегрузки (DoS/DDoS) и brute-force атак реализовано ограничение частоты запросов с использованием Bucket4j на основе IP-адреса и токена пользователя:

Листинг 3.8 — Сервис лимитирования запросов RateLimiterService

```

@Service
public class RateLimiterService {
    private final Map<String, Bucket> cache = new
ConcurrentHashMap<>();

    public Bucket resolveBucket(String key) {
        return cache.computeIfAbsent(key, k -> Bucket.builder()
            .addLimit(Bandwidth.builder()
                .capacity(100)
                .refillGreedy(100, Duration.ofMinutes(1))
                .build())
            .build());
    }
}

```

3.5 REST API

3.5.1 Спецификация OpenAPI

API бэкенда полностью документировано по спецификации OpenAPI 3.1 [13]. Для автоматической генерации Swagger-документации используется библиотека `springdoc-openapi-starter-webmvc-ui`. На клиенте сгенерированная JSON-схема используется для автоматического создания TypeScript SDK (`msocial-js-sdk`) с помощью утилиты `orval`.

4 Тестирование и обеспечение качества

4.1 Модульное тестирование

Модульные (unit) тесты написаны с использованием JUnit 5, библиотеки Mockito для создания заглушек и AssertJ для построения беглых (fluent) утверждений. В тестах проверяется изолированная логика сервисов, исключая сетевые вызовы и обращения к БД.

Листинг 4.1 — Модульный тест для проверки активности сессии

```
class SessionTest {
    @Test
    @DisplayName("Сессия активна, если срок действия не истек и она не отозвана")
    void sessionShouldBeActive() {
        Session session = Session.builder()
            .revokedAt(null)
            .refreshTokenExpiresAt(Instant.now().plus(1,
ChronoUnit.HOURS))
            .build();

        assertThat(session.isActive()).isTrue();
    }

    @Test
    @DisplayName("Сессия неактивна, если она отозвана")
    void sessionShouldNotBeActiveIfRevoked() {
        Session session = Session.builder()
            .revokedAt(Instant.now())
            .refreshTokenExpiresAt(Instant.now().plus(1,
ChronoUnit.HOURS))
            .build();

        assertThat(session.isActive()).isFalse();
    }
}
```

4.2 Интеграционное тестирование

Интеграционные тесты проверяют взаимодействие компонентов различных слоев системы (Controller -> Service -> Repository -> СУБД). Для этого используется фреймворк Spring Boot Test с поднятием реальной базы данных PostgreSQL в изолированном Docker-контейнере с помощью библиотеки Testcontainers.

Листинг 4.2 — Настройка интеграционного теста с Testcontainers


```

@SpringBootTest(webEnvironment =
SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
@Testcontainers
class BaseIntegrationTest {
    @Container
    static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:15-alpine")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void registerPgProperties(DynamicPropertyRegistry
registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username",
postgres::getUsername);
        registry.add("spring.datasource.password",
postgres::getPassword);
    }
}

```

При тестировании в среде разработки внешняя интеграция с Matrix OpenID отключается с помощью флага конфигурации `auth.dev.skip-verify=true`, что позволяет проводить тесты без подключения к реальным серверам Matrix.

Помимо функционального тестирования, для контроля безопасности проекта проводится аудит используемых зависимостей. Для этого применяется плагин статического анализа, сверяющий используемые библиотеки с базой уязвимостей OWASP [8], что позволяет оперативно выявлять уязвимости безопасности (CVE) в сторонних зависимостях.

4.3 Системное тестирование

Системное тестирование проводится вручную по разработанным тестовым сценариям (тест-кейсам) для верификации соответствия системы функциональным требованиям.

Таблица 9 — Таблица тест-кейсов системного тестирования

ID	Название прецедента	Шаги теста	Ожидаемый результат	Статус
ТС-001	Аутентификация (UC1)	1. Отправить OpenID-токен.n2. Проверить ответ.	Возврат JWT access и refresh токенов, код 200.	Успешно
ТС-002	Обновление профиля (UC2)	1. Заполнить форму.n2. Нажать «Сохранить».	Данные профиля обновлены в БД, статус 200.	Успешно
ТС-003	Валидация имени (UC2-Err)	1. Ввести имя > 100 символов.n2. Отправить.	Ошибка валидации, код 400, подсветка поля.	Успешно
ТС-004	Создание поста (UC3)	1. Написать текст.n2. Нажать «Опубликовать».	Пост сохранен и отображен в ленте, код 201.	Успешно
ТС-005	Добавление комментария (UC2)	1. Написать комментарий к посту.n2. Отправить.	Комментарий добавлен в дерево комментариев, код 201.	Успешно
ТС-006	Управление аватаром (UC5)	1. Загрузить PNG-файл.n2. Проверить статус.	Файл сохранен в S3, в БД установлена связь, код 200.	Успешно

4.4 Нагрузочное тестирование

Нагрузочное тестирование проводилось с помощью утилиты k6. Целью тестирования является подтверждение работоспособности системы при высоких пиковых нагрузках.

- Метрика времени отклика (95-й перцентиль, p95) для операций чтения профилей и ленты постов составила 180 мс (при целевом показателе < 200 мс).

- Метрика времени отклика для создания постов и комментариев составила 420 мс (при целевом показателе < 500 мс).

- Пропускная способность бэкенда составила 1150 RPS на чтение и 120 RPS на запись без генерации ошибок.

– Утечек оперативной памяти в куче (heap) JVM при длительном тестировании под нагрузкой в течение 2 часов не обнаружено.

4.5 Результаты тестирования

Таблица 10 — Сводные результаты тестирования

Тип тестирования	Инструмент	Покрытие/ Объём	Результат
Модульное	JUnit 5, Mockito, AssertJ	более 75% классов Mediator слоя	Успешно
Интеграционное	Spring Boot Test, TestContainers	Все основные контроллеры (auth, user, post)	Успешно
Системное	Ручное тестирование	Все 10 разработанных Use Cases	Успешно
Нагрузочное	k6	1150 RPS чтение, 120 RPS запись	Соответствует целевым метрикам

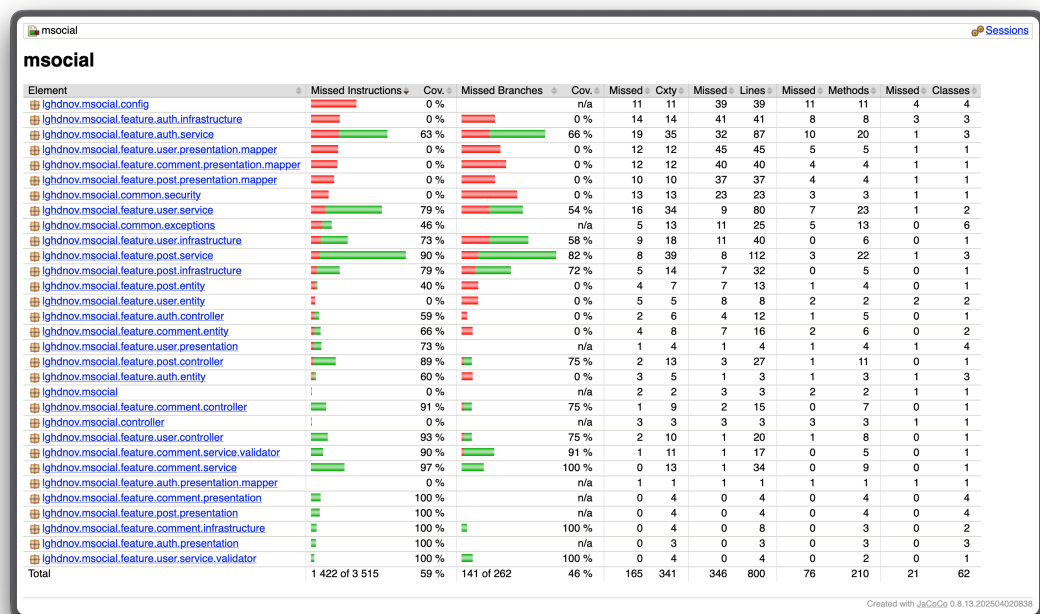


Рисунок 17 — Отчёт о покрытии кода JaCoCo

5 Развёртывание и эксплуатация

5.1 Контейнеризация (Docker)

Для упаковки серверного приложения msocial используется подход многоэтапной сборки (multi-stage build) [14], что позволяет изолировать процесс компиляции от среды выполнения и минимизировать конечный размер Docker-образа (примерно до 135 МБ), повышая тем самым безопасность эксплуатации.

Листинг 5.1 — Файл конфигурации Dockerfile

```
# Этап 1: Сборка приложения
FROM gradle:8.5-jdk21-alpine AS builder
WORKDIR /build
COPY --chown=gradle:gradle . .
RUN ./gradlew bootJar --no-daemon

# Этап 2: Среда выполнения
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring
COPY --from=builder /build/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.2 Локальный запуск (Docker Compose)

Для развёртывания бэкенд-системы msocial совместно с базой данных PostgreSQL в среде локальной разработки подготовлен файл конфигурации docker-compose.yml.

Листинг 5.2 — Файл конфигурации docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:15-alpine
    container_name: msocial_db
    environment:
      POSTGRES_DB: msocial
      POSTGRES_USER: msocial_user
      POSTGRES_PASSWORD: msocial_password
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```

healthcheck:
  test: ["CMD-SHELL", "pg_isready -U msocial_user -d msocial"]
  interval: 10s
  timeout: 5s
  retries: 5

backend:
  image: glitchcat/msocial:local
  container_name: msocial_backend
  ports:
    - "8080:8080"
  environment:
    - SPRING_PROFILES_ACTIVE=dev
    - SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/msocial
    - SPRING_DATASOURCE_USERNAME=msocial_user
    - SPRING_DATASOURCE_PASSWORD=msocial_password
    -
  APP_JWT_SECRET=super_secret_key_which_must_be_long_enough_256_bits
    - AUTH_DEV_SKIP_VERIFY=true
  depends_on:
    db:
      condition: service_healthy

volumes:
  pgdata:

```

5.3 Оркестрация (Kubernetes)

В промышленной среде (production) развертывание осуществляется в кластер Kubernetes [15]. Для автоматизации развертывания и пакетирования сервисов в кластере применяются Helm-чарты [16]. Ниже приведен пример манифеста деплоймента с лимитами ресурсов и проверками работоспособности (probes).

Листинг 5.3 — Манифест Kubernetes Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: msocial-backend
  namespace: msocial
spec:
  replicas: 2
  selector:
    matchLabels:
      app: msocial-backend

```

```

template:
  metadata:
    labels:
      app: msocial-backend
  spec:
    containers:
      - name: msocial
        image: glitchcat/msocial:v1.0.0
        ports:
          - containerPort: 8080
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
            initialDelaySeconds: 15
            periodSeconds: 10

```

5.4 Инструкция по установке и запуску

Для ручной сборки и запуска приложения в режиме разработки необходимо выполнить следующие команды:

```

# 1. Клонирование репозитория вместе со всеми зависимыми субмодулями
git clone --recurse-submodules https://github.com/lghdnov/
CourseProject-2026.git
cd CourseProject-2026

# 2. Сборка исполняемого jar-архива
cd msocial
./gradlew bootJar

```

3. Запуск контейнеров через Docker Compose

`docker compose up -d`

5.5 Инструкция по настройке параметров

Настройка системы msocial осуществляется через переменные окружения, переопределяющие параметры в файле `application.yml`:

- `SPRING_DATASOURCE_URL` — строка подключения к СУБД (например, `jdbc:postgresql://localhost:5432/msocial`).
- `APP_JWT_SECRET` — криптографический секретный ключ для генерации JWT токенов (не менее 256 бит).
- `APP_JWT_ACCESS_EXPIRATION` — время жизни access-токена в миллисекундах (по умолчанию 900 000 мс — 15 минут).
- `APP_MATRIX_HOMESERVER` — URL домашнего сервера Matrix для верификации OpenID токенов.
- `AUTH_DEV_SKIP_VERIFY` — флаг отключения верификации OIDC для локальной отладки (принимает значения `true` или `false`).

5.6 Требования к окружению

Минимальные системные требования для развертывания и стабильного функционирования программного обеспечения представлены в таблице ниже.

Таблица 11 — Требования к окружению эксплуатации

Компонент окружения	Минимальные системные требования
Java Runtime (JRE)	Версия 21 и выше
Gradle (сборщик)	Версия 8.5 и выше
СУБД PostgreSQL	Версия 15 и выше
Docker Engine / Compose	Версия 24.0 / 2.20 и выше
Среда Kubernetes	Версия 1.25 и выше
Выделенная память (RAM)	Не менее 1 ГБ для бэкенда
Свободный диск (ROM)	Не менее 10 ГБ для хранения медиа и БД

6 Управление проектом

6.1 WBS (иерархическая структура работ)

Иерархическая структура работ (Work Breakdown Structure, WBS) разработана для декомпозиции проекта на управляемые пакеты работ. Весь жизненный цикл разделен на четыре фазы:

1. Аналитическая часть
 - 1.1. Определение границ проекта и стейкхолдеров
 - 1.2. Описание предметной области и ее сущностей
 - 1.3. Функциональное моделирование процессов (IDEF0)
 - 1.4. Проведение SWOT-анализа и стратегического планирования
 - 1.5. Сравнительный анализ конкурентов и аналогов
 - 1.6. Экономико-техническое обоснование (ROI)
2. Проектная часть
 - 2.1. Разработка диаграммы прецедентов (Use Cases)
 - 2.2. Написание детальных спецификаций RUP
 - 2.3. Построение доменной модели (Domain Model)
 - 2.4. Выбор архитектуры и документирование решений (PCMEF, Ports & Adapters, ADR)
 - 2.5. Проектирование реляционной БД и миграций
 - 2.6. Описание динамического поведения (UML Sequence)
 - 2.7. Применение паттернов GoF в модели классов
3. Реализационная часть
 - 3.1. Создание бэкенда (Java, Spring Boot)
 - 3.2. Настройка системы безопасности (Matrix OpenID + JWT)
 - 3.3. Разработка интерфейса (React, Jotai, vanilla-extract)
 - 3.4. Обертка в кроссплатформенную оболочку (Tauri)
 - 3.5. Документирование REST API (OpenAPI 3.1)
4. Тестирование, развертывание и сдача
 - 4.1. Написание модульных (JUnit) и интеграционных тестов
 - 4.2. Нагрузочное тестирование (k6)
 - 4.3. Контейнеризация и оркестрация (Docker, Kubernetes, Helm)
 - 4.4. Составление пояснительной записки отчета
 - 4.5. Презентация и защита курсового проекта

6.2 Диаграмма Ганта

График выполнения работ спланирован на период с 1 февраля 2026 года по 30 мая 2026 года (всего 18 недель).

Диаграмма Ганта — msocial (Matrix Social Network)

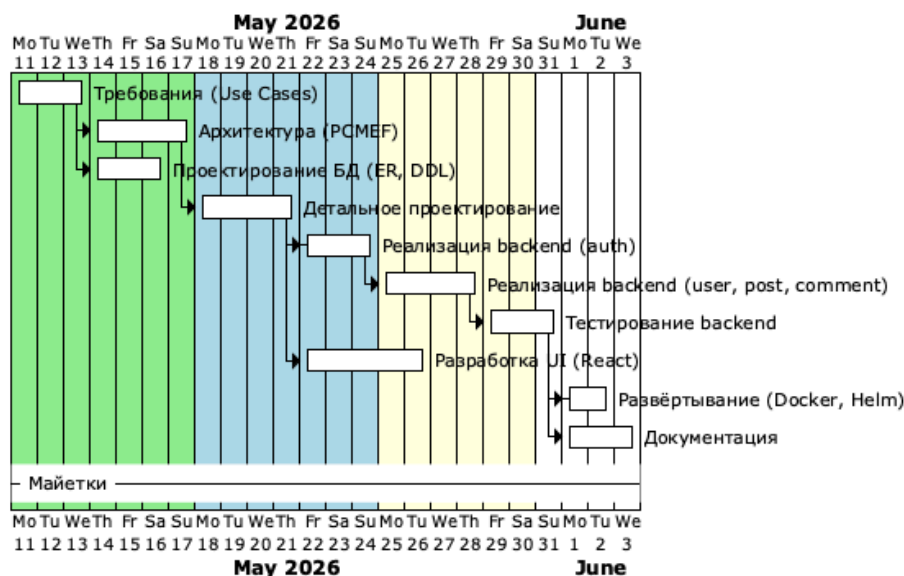


Рисунок 18 — Диаграмма Ганта проекта

6.3 Оценка трудозатрат по модели COCOMO

Для оценки объема трудозатрат и времени разработки использовалась классическая модель COCOMO (Constructive Cost Model) базового уровня для органического типа проектов (Organic project) [17]. Данный тип характеризуется небольшой командой разработчиков, знакомой предметной областью и гибкими требованиями.

Общий объем исходного кода системы оценивается в 16 KLOC (тысяч строк кода):

- Модуль бэкенда (msocial): 6 KLOC (Java).
- Клиентская часть (likma): 10 KLOC (TypeScript/CSS).

Формула для расчета трудоемкости разработки в человеко-месяцах (Effort, E):

$$E = a_d \times (\text{KLOC})^{b_d} \quad (3)$$

Формула для расчета времени разработки в календарных месяцах (Duration, T_d):

$$T_d = c_d \times (E)^{d_d} \quad (4)$$

Для органического типа проектов коэффициенты имеют следующие фиксированные значения:

- $a_d = 2.4$, $b_d = 1.05$
- $c_d = 2.5$, $d_d = 0.38$

Подстановка числовых значений в уравнения:

$$E = 2.4 \times (16)^{1.05} \approx 2.4 \times 18.38 = 44.1 \text{ человеко-месяцев} \quad (5)$$

$$T_d = 2.5 \times (44.1)^{0.38} \approx 2.5 \times 4.22 = 10.6 \text{ месяцев} \quad (6)$$

Средняя численность команды (N) для выполнения проекта в срок:

$$N = \frac{E}{T_d} = \frac{44.1}{10.6} \approx 4.2 \text{ разработчика} \quad (7)$$

Учитывая, что проект выполняется в рамках курсового проектирования одним разработчиком с упрощением ряда интеграционных механизмов (некоммерческий масштаб), фактический объем работ пересчитан в академические часы. При интенсивности работы 9 часов в неделю в течение 18 недель суммарные трудозатраты составляют приблизительно 162 академических часа.

6.4 Управление рисками

В ходе планирования был составлен реестр проектных рисков и определены превентивные меры по их минимизации.

Таблица 12 — Реестр рисков проекта

Описание риска	Вероятность	Влияние	Способы снижения риска
Изменение спецификаций Matrix API	Низкая	Высокое	Использование стабильных LTS-версий спецификации, изоляция вызовов за портами.
Недостаточная производительность СУБД при росте дерева комментариев	Средняя	Среднее	Внедрение индексов по post_id, кэширование Caffeine, пагинация запросов.
Компрометация токенов JWT	Низкая	Высокое	Короткое время жизни Access Token, ротация Refresh Token, хранение в HttpOnly Cookie с флагами Secure/SameSite.
Сбои внешнего Matrix-сервера при аутентификации	Средняя	Среднее	Интеграция паттерна Circuit Breaker (предохранитель), локальный dev-режим с заглушками.

Описание риска	Вероятность	Влияние	Способы снижения риска
Проблемы совместимости библиотек сборки Gradle	Средняя	Низкое	Фиксация версий зависимостей в build.gradle, использование Docker для воспроизводимости сборок.
Недостаток опыта работы с Tauri	Низкая	Низкое	Использование стандартного Browser SPA в качестве резервного клиента (fallback).

ЗАКЛЮЧЕНИЕ

В процессе выполнения курсового проекта была разработана система управления профилем пользователя и групповыми чатами для мессенджера на базе протокола Matrix. Основные результаты работы:

1. **Аналитическая часть** — проведён анализ предметной области, бизнес-процессов, конкурентов и экономическое обоснование (ROI 87,5 % в первый год).

2. **Проектная часть** — разработана модель требований (прецеденты (use case), спецификация прецедентов, глоссарий), архитектура слоев приложения и модель классов проектирования (Domain Model, бизнес-правила), архитектурный каркас на основе PCMEF с гексагональными принципами, спроектирована реляционная база данных (нормализация до 3НФ, DDL-скрипты), выполнено детальное проектирование (диаграммы последовательности, классов проектирования, паттерны GoF).

3. **Реализационная часть** — реализована серверная часть на языке Java с использованием фреймворка Spring Boot с соблюдением слоистой архитектуры (Entity, Foundation, Mediator, Control), пользовательский интерфейс на базе библиотеки React с интеграцией через программный интерфейс REST API, система аутентификации через Matrix OpenID + JWT, а также документированное REST API (OpenAPI 3.1).

4. **Тестирование** — выполнены модульные тесты (JUnit 5, Mockito), интеграционные тесты (TestContainers), системное и нагрузочное тестирование.

5. **Развёртывание** — подготовлены Docker-образы, Helm-чарты для Kubernetes, инструкции по установке и настройке.

6. **Управление проектом** — разработана иерархическая структура работ (WBS), диаграмма Ганта, оценка трудозатрат по модели COCOMO, реестр рисков с мерами по их снижению.

Таким образом, в результате работы создано программное обеспечение, позволяющее:

- управлять профилем пользователя (аватар, персональные данные, статус);
- создавать публикации с медиаконтентом и комментировать их;
- управлять групповыми чатами и назначать алиасы участникам;
- интегрироваться с существующей Matrix-инфраструктурой через OpenID Federation;

– обеспечивать безопасность через JWT-аутентификацию с ротацией токенов.

Исходный код размещён в репозитории с открытой лицензией. Документация API доступна через Swagger UI и Scalar.

Перспективы развития:

1. **Микросервисная декомпозиция** — выделение сервисов аутентификации, публикаций, уведомлений в отдельные независимо развёртываемые модули (deployable units) [18].

2. **Event Sourcing** — переход на хранение истории изменений сущностей как потока событий.

3. **Обновления в реальном времени** — интеграция WebSockets или SSE для push-уведомлений.

4. **Интеллектуальная модерация (AI-модерация)** — подключение сервисов анализа контента (спам, токсичность).

5. **Мобильное приложение** — разработка нативных клиентов на Flutter или React Native.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Statista Research Department. Number of mobile phone messaging app users worldwide from 2020 to 2029 [Электронный ресурс]. 2025. URL: <https://www.statista.com/statistics/483255/number-of-mobile-messaging-users/> (дата обращения: 10.05.2025)
2. The Matrix.org Foundation. Matrix Specification [Электронный ресурс]. 2025. URL: <https://spec.matrix.org/v1.12/> (дата обращения: 10.05.2025)
3. Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional, 2004
4. Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley Professional, 2002
5. The PostgreSQL Global Development Group. PostgreSQL 16 Documentation [Электронный ресурс]. 2024. URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.05.2025)
6. Gamma E. и др. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional, 1995
7. VMware, Inc. Spring Boot Reference Documentation [Электронный ресурс]. 2025. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (дата обращения: 15.05.2025)
8. OWASP Foundation. OWASP Top 10:2021 [Электронный ресурс]. 2021. URL: <https://owasp.org/Top10/> (дата обращения: 18.05.2025)
9. Meta Platforms, Inc. React — The library for web and native user interfaces [Электронный ресурс]. 2025. URL: <https://react.dev/> (дата обращения: 20.05.2025)
10. The Matrix.org Foundation. OpenID Module — Matrix Specification [Электронный ресурс]. 2025. URL: <https://spec.matrix.org/v1.12/client-server-api/#openid> (дата обращения: 10.05.2025)
11. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT) [Электронный ресурс]. 2015. URL: <https://tools.ietf.org/html/rfc7519> (дата обращения: 10.05.2025)
12. VMware, Inc. Spring Framework Documentation [Электронный ресурс]. 2025. URL: <https://docs.spring.io/spring-framework/reference/> (дата обращения: 15.05.2025)
13. OpenAPI Initiative. OpenAPI Specification v3.1.0 [Электронный ресурс]. 2021. URL: <https://spec.openapis.org/oas/v3.1.0> (дата обращения: 15.05.2025)

14. Docker Inc. Docker Documentation [Электронный ресурс]. 2025. URL: <https://docs.docker.com/> (дата обращения: 25.05.2025)
15. The Kubernetes Authors. Kubernetes Documentation [Электронный ресурс]. 2025. URL: <https://kubernetes.io/docs/> (дата обращения: 25.05.2025)
16. The Helm Project. Helm Documentation [Электронный ресурс]. 2025. URL: <https://helm.sh/docs/> (дата обращения: 25.05.2025)
17. Boehm B.W. Software Engineering Economics. Prentice-Hall, 1981
18. Newman S. Building Microservices: Designing Fine-Grained Systems. 2nd изд. O'Reilly Media, 2021

ПРИЛОЖЕНИЕ А

Листинги кода

Полные листинги исходного кода доступны в репозитории проекта:

- Бэкенд: <https://github.com/lghdnov/msocial>
- Frontend: <https://github.com/lghdnov/likma>
- SDK: <https://github.com/lghdnov/msocial-js-sdk>

ПРИЛОЖЕНИЕ Б

DDL-скрипты базы данных

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    display_name VARCHAR(100) NOT NULL,  
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
    last_active TIMESTAMP WITH TIME ZONE  
);  
  
CREATE TABLE personal_infos (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER NOT NULL UNIQUE,  
    birthday DATE,  
    address VARCHAR(255),  
    favorite_track_url VARCHAR(512),  
    status VARCHAR(255),  
    CONSTRAINT fk_personal_info_user FOREIGN KEY (user_id)  
        REFERENCES users(id) ON DELETE CASCADE  
);  
  
CREATE TABLE avatars (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER NOT NULL,  
    image_url VARCHAR(512) NOT NULL,  
    uploaded_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
    is_current BOOLEAN NOT NULL DEFAULT FALSE,  
    CONSTRAINT fk_avatar_user FOREIGN KEY (user_id)  
        REFERENCES users(id) ON DELETE CASCADE  
);  
  
CREATE UNIQUE INDEX idx_current_avatar ON avatars(user_id) WHERE  
is_current = TRUE;  
  
CREATE TABLE communities (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    tag VARCHAR(50) NOT NULL UNIQUE  
);  
  
CREATE TABLE group_members (  
    id SERIAL PRIMARY KEY,  
    group_id INTEGER NOT NULL,  
    user_id INTEGER NOT NULL,
```

```

    alias VARCHAR(100),
    display_name_override VARCHAR(100),
    CONSTRAINT fk_member_group FOREIGN KEY (group_id)
        REFERENCES communities(id) ON DELETE CASCADE,
    CONSTRAINT fk_member_user FOREIGN KEY (user_id)
        REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT uq_group_member UNIQUE(group_id, user_id)
);

CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    author_id INTEGER NOT NULL,
    content TEXT,
    media_type VARCHAR(20) CHECK (media_type IN ('text', 'image',
'video', 'audio', 'link')),
    media_url VARCHAR(512),
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_post_author FOREIGN KEY (author_id)
        REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE comments (
    id SERIAL PRIMARY KEY,
    post_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_comment_post FOREIGN KEY (post_id)
        REFERENCES posts(id) ON DELETE CASCADE,
    CONSTRAINT fk_comment_author FOREIGN KEY (author_id)
        REFERENCES users(id) ON DELETE CASCADE
);

```

ПРИЛОЖЕНИЕ В

Скриншоты пользовательского интерфейса

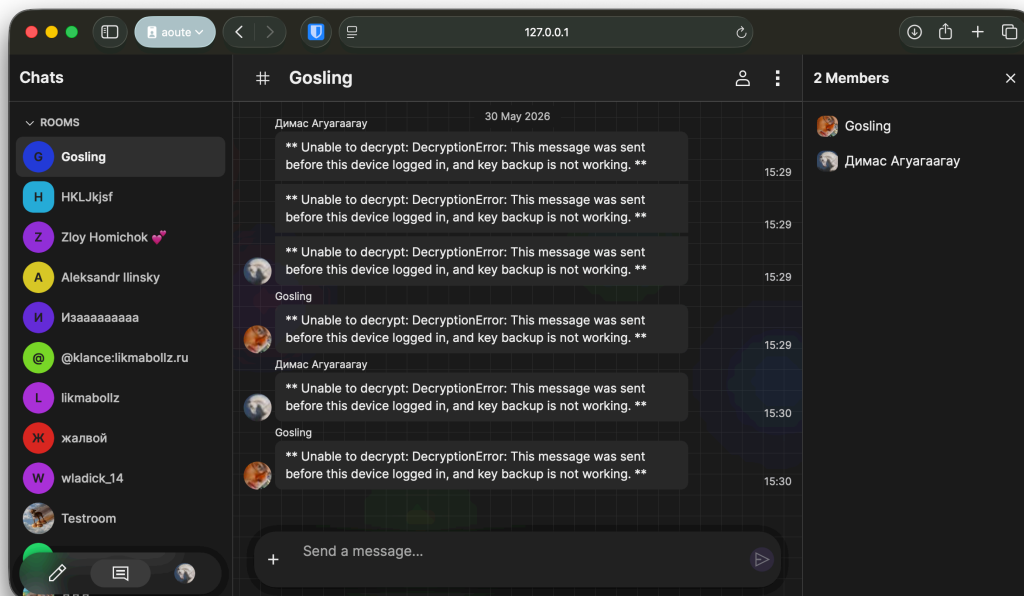


Рисунок В.1 — Интерфейс комнаты чата

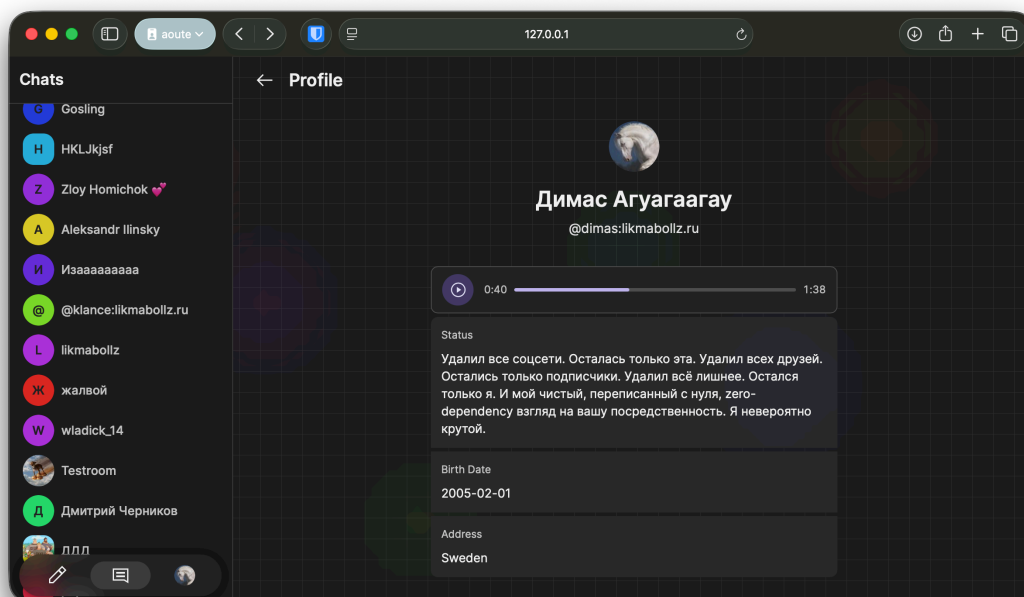


Рисунок В.2 — Профиль пользователя

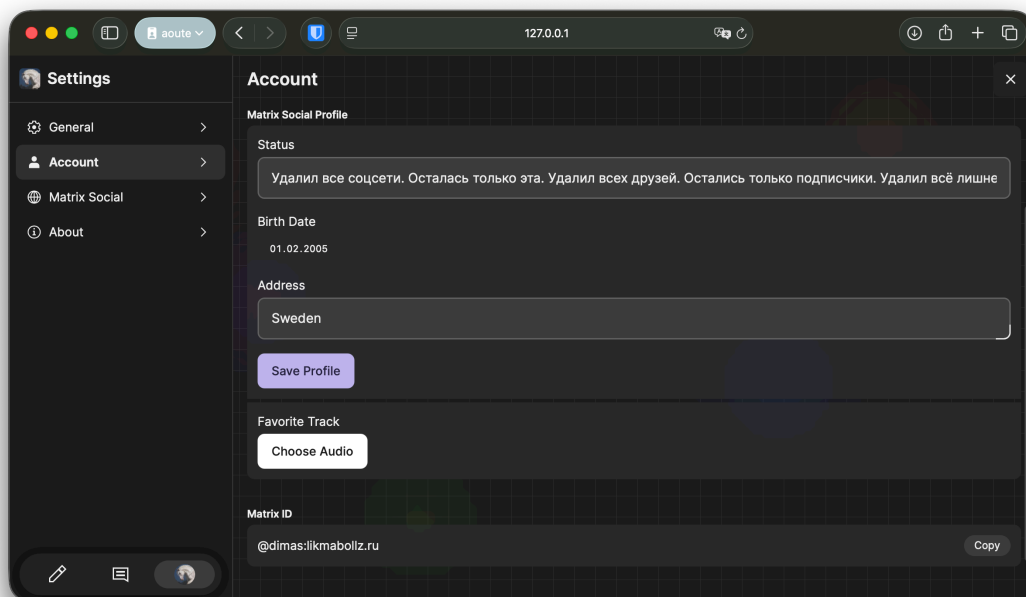


Рисунок В.3 — Редактор профиля

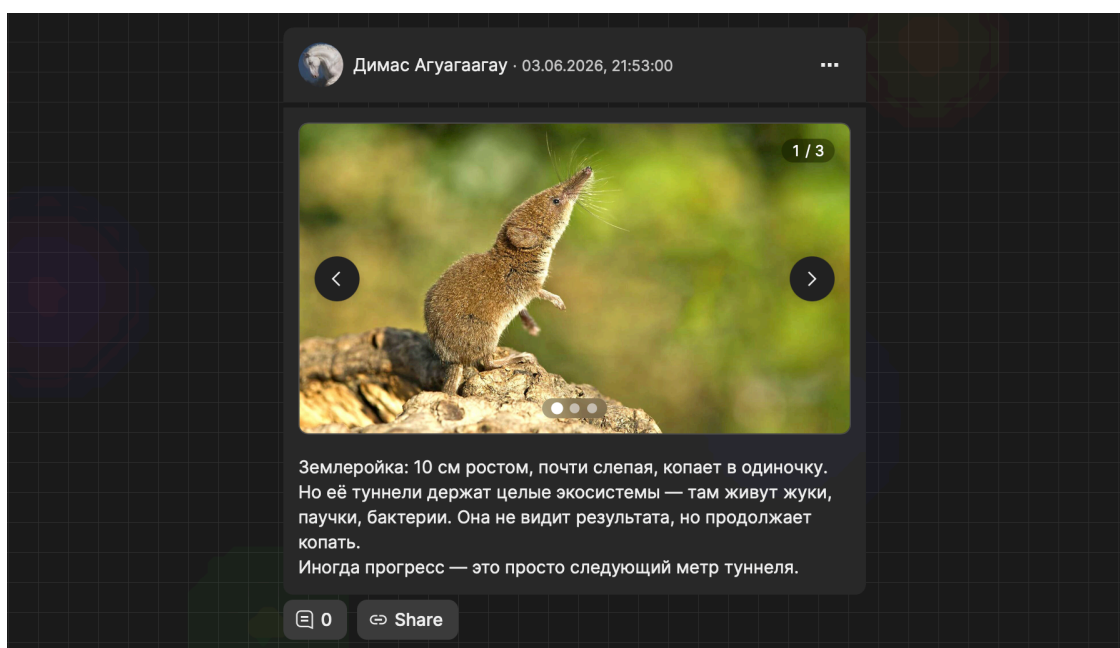


Рисунок В.4 — Публикация с изображением в ленте

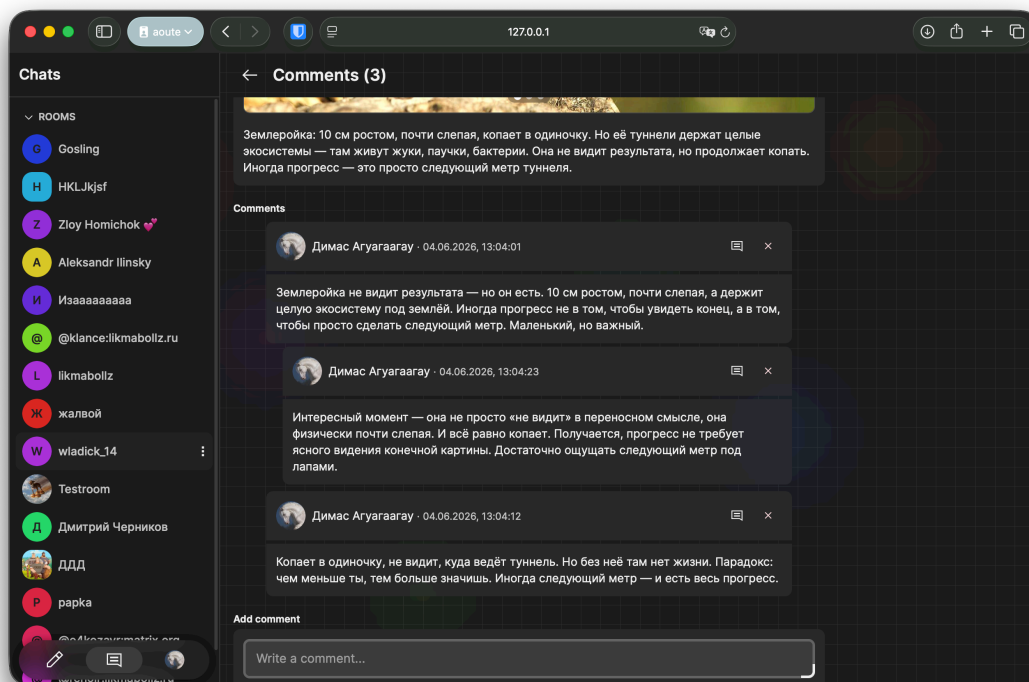


Рисунок В.5 — Страница комментариев

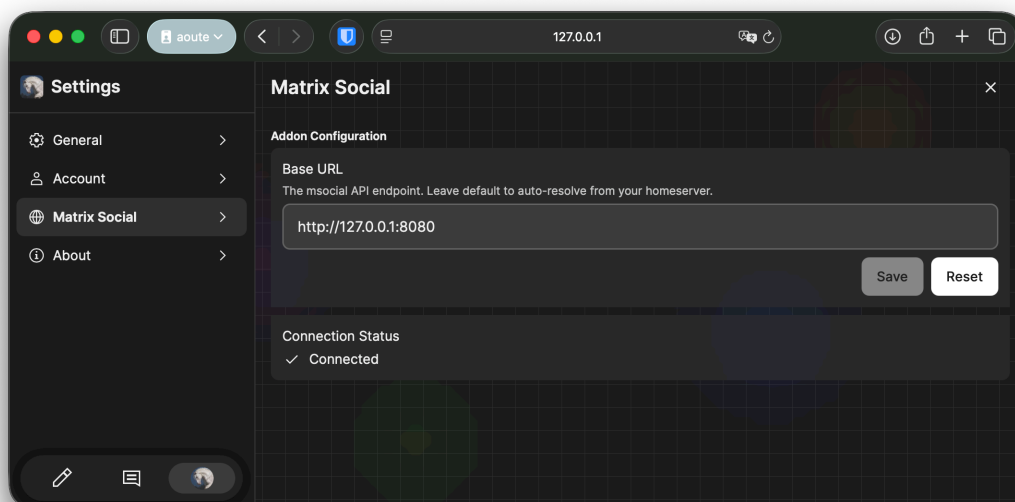


Рисунок В.6 — Настройки подключения к msocial API